

តិចនិចសរសេរកូដកម្រិតខ្ពស់
គំរូនៃការរៀបចំកូដ

សម្រាប់វិស្វករកុំព្យូទ័រមានចំណេះដឹងលើការសរសេរកូដលំនាំ OOP
និង ចេះប្រើប្រាស់ UML

Advanced Programming Technique

Design Pattern

អនុបណ្ឌិត ប៊ិន ប៊ុនធីល

ខែកញ្ញា ឆ្នាំ២០១៣

Version 1.0

ពុំមានការរក្សាសិទ្ធិអ្វីទេ សូមចាត់ចែង ប្រើប្រាស់ អាស្រ័យផលលើស្នាដៃនេះជាប្រយោជន៍តាមតែអាចធ្វើទៅបាន
ដោយពុំចាំបាច់សុំការអនុញ្ញាត ឬបញ្ជាក់ឈ្មោះអ្នកនិពន្ធទេ។

អារម្ភកថា

ជាបឋមខ្ញុំសូមលើកហត្ថទាំងពីរបង្គំសំពះដឹងគុណចំពោះអ្នកមានគុណទាំងទ្វេ ដែលលោកបានផ្តល់កំណើត និងតែងបានយកចិត្តទុកដាក់បីបាច់ថែរក្សារូបខ្ញុំតាំងពីតូចត្រីតូចមក សូមសំដែងការដឹងគុណចំពោះលោកឪពុកក្មេក អ្នកម្តាយក្មេក ពូ ប្អូនស្រី និងប្រពន្ធខ្ញុំ ដែលជួយមើលថែកូនប្រុស **ឈិន ធីលការុង** ក្នុងពេលខ្ញុំអង្គុយគិតសរសេរសៀវភៅនេះមិនបានជួយអ្វី និងសូមសំដែងការដឹងគុណយ៉ាងជ្រាលជ្រៅចំពោះលោកគ្រូ អ្នកគ្រូ សាស្ត្រាចារ្យ ទាំងអស់ ដែលតែងបានប្រឹងអស់ពីកម្លាំងកាយចិត្តអប់រំណែនាំរូបខ្ញុំរៀនរហូតមក ។ ទាំងអស់នេះគឺជាមូលដ្ឋានដ៏សំខាន់ធ្វើឱ្យរូបខ្ញុំអាចតាក់តែងសៀវភៅនេះឡើងដើម្បីជាប្រយោជន៍ដល់កូនខ្មែរជំនាន់ក្រោយ។

សូមអរគុណជាពិសេសចំពោះលោកសាស្ត្រាចារ្យពីររូបនៃសាកលវិទ្យាល័យភូមិន្ទភ្នំពេញ ក្នុងនោះមានលោកសាស្ត្រាចារ្យ **ជី ក្នុង** និង លោកសាស្ត្រាចារ្យ **តាំង ប៉េងស្រេង** ដែលបានបង្ហាត់បង្ហាញខ្ញុំលើកំរនៃការរៀបចំកូដនេះ។

សូមថ្លែងអំណរគុណដល់មិត្តភក្តិទាំងអស់ដែលជួយគាំទ្រ និងកែលម្អចំណុចខ្វះខាតដែលមាននៅក្នុងសៀវភៅនេះ។

ក្នុងការរៀបចំចងក្រងស្នាដៃនេះឡើង ដោយក្តីស្រឡាញ់ចំពោះអក្សរសាស្ត្រជាតិ ខ្ញុំបានព្យាយាមមិនប្រើបច្ចេកសព្ទបរទេសច្រើនពេកទេ។ ខ្ញុំបានបកប្រែពាក្យទាំងនោះមកជាខ្មែរដោយផ្អែកលើអត្ថន័យ និងការប្រើប្រាស់របស់វា ហើយដើម្បីងាយចងចាំផង។

តែទោះបីជាមានការខិតខំសិរិសោធន៍យ៉ាងណាក្តី ខ្ញុំគិតថាកំហុសឆ្គងដោយសារការយល់រាក់សេចក្តី បកស្រាយអត្ថន័យល្អៀងភាពដើមប្រាកដជាមាន ហេតុនេះខ្ញុំសូមរីករាយស្វាគមន៍ដល់ការរិះគន់ស្ថាបនាទាំងឡាយដើម្បីញ៉ាំងឱ្យស្នាដៃនេះកាន់តែមានសុក្រឹតភាព និងសារប្រយោជន៍ដល់កូនខ្មែរទាំងអស់។

រាជធានីភ្នំពេញ ថ្ងៃទី២៧ ខែកញ្ញា ឆ្នាំ២០១៣

បិន ប៊ុនធីល

ទំនាក់ទំនងអ្នកនិពន្ធ

សារអេឡិចត្រូនិច៖

bunchhel@hotmail.com

bunchhel@yahoo.com

bunchhel@gmail.com

ទូរស័ព្ទដៃ

០១៧ ៨៧៧៧៧៨

សេចក្តីផ្តើម	4
តើសៀវភៅនេះសម្រាប់នរណា?	4
របកគំហើញកំរនៃការរៀបចំកូដ និងអត្ថប្រយោជន៍	9
កំរនៃការរៀបចំកូដតាមប្រភេទ និងការប្រើប្រាស់របស់វា.....	10
ប្រភេទកំរនៃបែបបង្កើត (Creational Pattern)	12
កំរនៃបែបឯកមជ្ឈិម (Singleton Pattern).....	13
កំរនៃបែបអត្តនិមិត្ត (Prototype Pattern).....	17
កំរនៃបែបផលិតកម្ម (Factory Pattern)	22
កំរនៃបែបស្ថាបនិក (Builder Pattern)	26
ប្រភេទកំរនៃបែបចាត់ចែង (Structural Pattern)	31
កំរនៃបែបផ្សិតតម្រូវ (Adapter Pattern)	32
កំរនៃបែបអន្តរទំនាក់ទំនង (Bridge Pattern)	35
កំរនៃបែបបន្ថែមលម្អ (Decorator Pattern).....	40
កំរនៃបែបសមាសចម្រុះ (Composite Pattern)	44
ប្រភេទកំរនៃបែបប្រព្រឹត្ត (Behavioral Pattern)	48
កំរនៃបែបវដ្តដំណើរ (Iterator Pattern).....	49
កំរនៃបែបស្វ័យគ្រប់គ្រង (Command Pattern).....	53
កំរនៃបែបរក្សាចំណាំ (Memento Pattern)	57
កំរនៃបែបសណ្ឋានលក្ខខណ្ឌ (State Pattern)	62
កំរនៃបែបឧទ្ទាតពុម្ព (Template Method Pattern)	66
កំរនៃបែបយុទ្ធសាស្ត្រ (Strategy Pattern)	70
កំរនៃបែបសង្កេតការណ៍ (Observer Pattern).....	74
កំរនៃបែបទេសចរណ៍ (Visitor Pattern).....	79
កំរនៃបែបបំណកប្រែ (Interpreter Pattern)	83
កំរនៃបែបសង្វាក់ទំនួលខុសត្រូវ (Chain of Responsibility Pattern)	87
កំរនៃបែបសន្លាតកម្ម (Model View Controller Pattern).....	92
វាចាអ្នកនិពន្ធ.....	97
ការស្វ័យសិក្សា.....	97
ការតម្កល់ខ្លួនជាប្រយោជន៍ដល់ជនដទៃ	97
តម្កល់ចិត្តរីករាយចំពោះអ្វីដែលខ្លួនកំពុងមាន	97

សេចក្តីផ្តើម

តើសៀវភៅនេះសម្រាប់នរណា ?

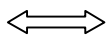
បើសិនអ្នកធ្លាប់ស្គាល់ និងចេះប្រើប្រាស់នូវ OOP UML VB.NET Encapsulation Inheritance Polymorphism Object Class Interface ឬមានចំណាប់អារម្មណ៍លើសិល្បៈនៃការចាត់ចែងការងារ នោះសៀវភៅនេះប្រហែលជាសម្រាប់អ្នកហើយ។

រំលឹក UML (Unified Modeling Language)

១. និមិត្តសញ្ញា និងការប្រើប្រាស់មួយចំនួន

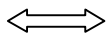
១.១ ការប្រកាសអញ្ញត្តិ និងប្រកាស Function

-name:String



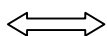
Private name As String

-age:Integer



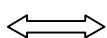
Private age As Integer

+SetName(value:String)



Public Sub SetName(ByVal value As String)
End Sub

+GetName():String



Public Function GetName() As String
Return "String"
End Function

១.២ Modifier

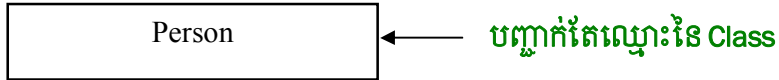
— តំណាងឱ្យ Private

+ តំណាងឱ្យ Public

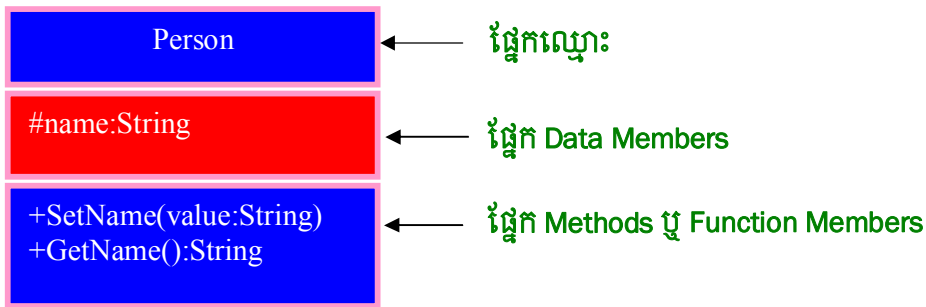
តំណាងឱ្យ Protected

១.៣ Class

Class អាចត្រូវបានតំណាងដោយនិមិត្តសញ្ញាសង្ខេបជាចតុកោណកែង៖



Class ក៏អាចតំណាងដោយនិមិត្តសញ្ញាលម្អិតជាងនេះ៖



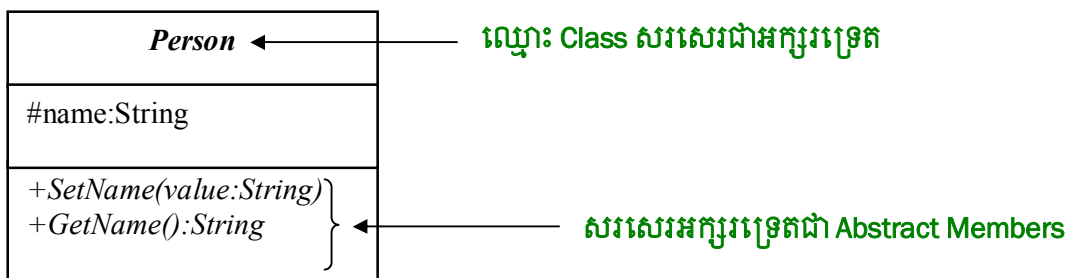
និមិត្តសញ្ញាខាងលើស្មើនឹងការសរសេរ Code ដូចខាងក្រោម៖

```
'Class
Public Class Person
    Protected name As String

    Public Sub SetName(ByVal value As String)
    End Sub

    Public Function GetName() As String
    End Function
End Class
```

១.៤ Abstract Class



និមិត្តសញ្ញាខាងលើស្មើនឹងការសរសេរ Code ដូចខាងក្រោម៖

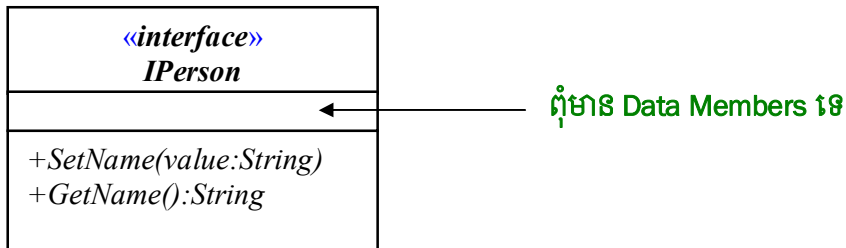
```
Public MustInherit Class Person
    Protected name As String

    Public MustOverride Sub SetName(ByVal value As String)
    Public MustOverride Function GetName() As String
End Class
```

Keyword **MustInherit** សម្គាល់ថា Class Person ពុំអនុញ្ញាតឱ្យ Instantiate បានទេ គឺមានតែ Class ជំនាន់ក្រោយ (Child Class ឬ Sub Class) របស់វាប៉ុណ្ណោះទើបអាចត្រូវបាន Instantiate បាន។

Keyword **MustOverride** បញ្ជាក់ថា Class ជំនាន់ក្រោយត្រូវតែ Implement ទៅលើ Sub Procedure ឬ Function ទាំងនោះ។

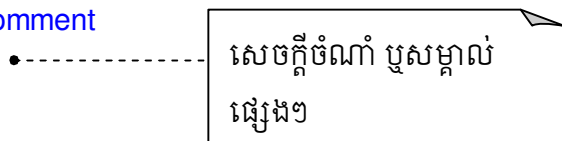
១.៥ Interface



និមិត្តសញ្ញាខាងលើស្នើនឹងការសរសេរ Code ដូចខាងក្រោម៖

```
Public Interface IPerson
    Sub SetName(ByVal value As String)
    Function GetName() As String
End Interface
```

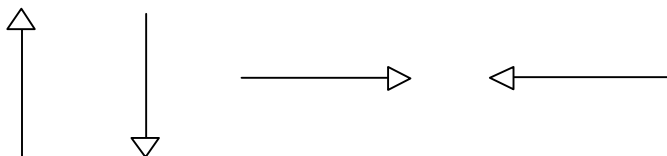
១.៦ Comment



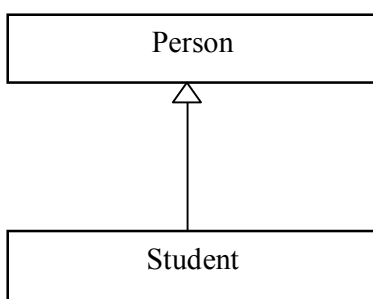
២. ទំនាក់ទំនងរវាងមួយចំនួន (Relationship)

២.១ Inheritance

Inheritance ជាទំនាក់ទំនងពី Class ជំនាន់ក្រោយ (Child Class ឬ Sub Class) ទៅ Class ដើម (Parent Class ឬ Super Class)។ Inheritance តំណាងដោយបណ្តានិមិត្តសញ្ញាខាងក្រោម៖



ឧទាហរណ៍៖



និមិត្តសញ្ញាខាងលើស្មើនឹងការសរសេរ Code ដូចខាងក្រោម៖

```
Public Class Person
End Class
```

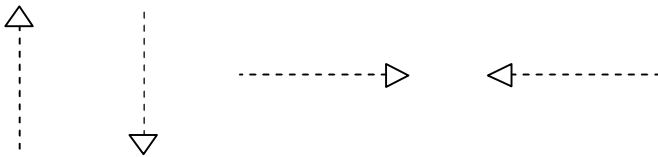
```
Public Class Student
Inherits Person
```

```
End Class
```

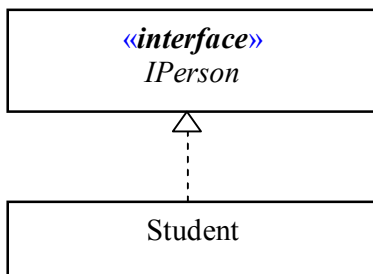
២.២ Implementation (ប្រើជាមួយ Interface)

នៅពេល Class ប្រើប្រាស់ Interface ណាមួយ នោះគេនិយាយថា Class Implement Interface នោះ។

Implementation តំណាងដោយបណ្តានិមិត្តសញ្ញាខាងក្រោម៖



ឧទាហរណ៍៖



និមិត្តសញ្ញាខាងលើស្មើនឹងការសរសេរ Code ដូចខាងក្រោម៖

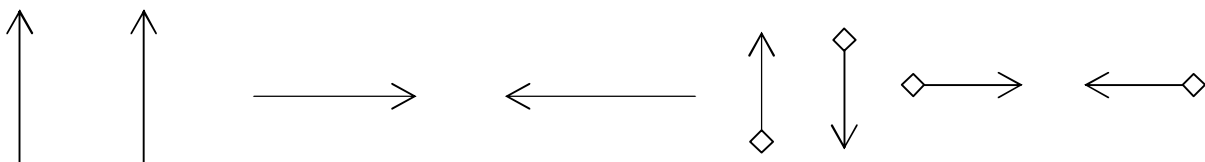
```
Public Interface IPerson
End Interface
```

```
Public Class Student
Implements IPerson
```

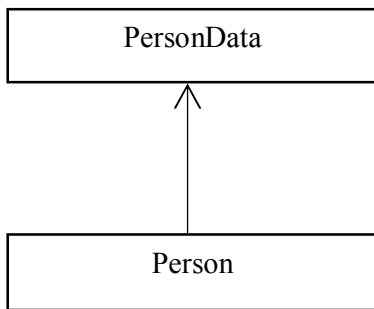
```
End Class
```

២.៣ Composition

Composition ជាទំនាក់ទំនងដែលកើតឡើងដោយសារ Object នៃប្រភេទ Class មួយត្រូវបានប្រកាសនៅក្នុង Class មួយទៀត។ Composition តំណាងដោយបណ្តានិមិត្តសញ្ញាខាងក្រោម៖



ឧទាហរណ៍១៖



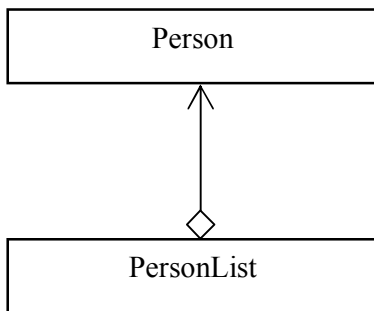
និមិត្តសញ្ញាខាងលើស្នើនឹងការសរសេរ Code ដូចខាងក្រោម៖

```
Public Class PersonData
End Class
```

```
Public Class Person
  Private pdata As PersonData
End Class
```

មានការប្រកាស Object ប្រភេទ
PersonData នៅក្នុង Class Person

ឧទាហរណ៍២៖



និមិត្តសញ្ញាខាងលើស្នើនឹងការសរសេរ Code ដូចខាងក្រោម៖

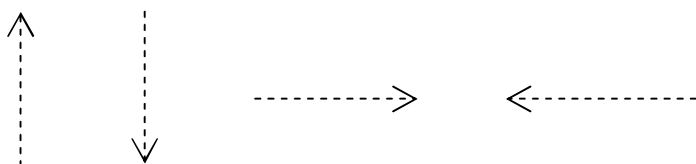
```
Public Class Person
End Class
```

```
Public Class PersonList
  Inherits List(Of Person)
End Class
```

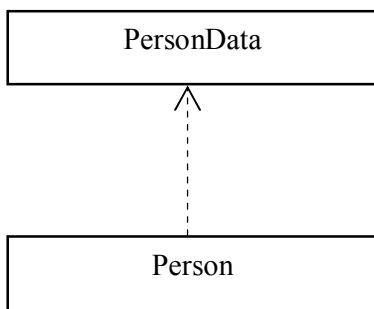
PersonList អាចផ្ទុកទៅដោយ Object
ជាច្រើននៃប្រភេទ Class Person

២.៤ Association

Association ជាទំនាក់ទំនងកើតឡើងដោយសារ Object នៃ Class មួយត្រូវបាន Return តាមរយៈ Function ណាមួយនៃ Class មួយទៀត។ Association តំណាងដោយបណ្តានិមិត្តសញ្ញាខាងក្រោម៖



ឧទាហរណ៍៖



និមិត្តសញ្ញាខាងលើស្នើនឹងការសរសេរ Code ដូចខាងក្រោម៖

```
Public Class PersonData
End Class
```

```
Public Class Person
```

```
Public Function GetData() As PersonData
Return New PersonData()
```

```
End Function
End Class
```

មានការ Return តម្លៃជា Object ប្រភេទ
PersonData នៅក្នុង Class Person

បេកគំហើញគំរូនៃការរៀបចំកូដ និងអត្ថប្រយោជន៍

គំរូនៃការរៀបចំកូដរកឃើញដោយ The “Gang of Four” or “GoF” ដែលមាន Erich Gamma, Richard Helm, Ralph Johnson និង John Vlissides ។ សៀវភៅរបស់ពួកគេមានចំណងជើងថា “Design Patterns: Elements of Reusable Object-Oriented Software” ដែលបោះពុម្ពដំបូង នៅឆ្នាំ ១៩៩៤ គឺជាសៀវភៅដែលល្បីល្បាញនិងទាក់ទាញបំផុតសម្រាប់វិស្វករកុំព្យូទ័រ ដែលលក់ជាច្រើនរហូតដល់ ៥សែនក្បាល ជាភាសាអង់គ្លេស និង ភាសាចំនួន ១៣ ផ្សេងទៀត។ រហូតមកទល់ខែមីនា ឆ្នាំ ២០១២ សៀវភៅនេះត្រូវបានបោះពុម្ពជាលើកទី ៤០ ហើយ។

គំរូនៃការរៀបចំកូដពិតជាមានអត្ថប្រយោជន៍ដ៏សំខាន់ សម្រាប់ការប្រើប្រាស់កូដចាស់ឡើងវិញ វាអនុញ្ញាតឱ្យវិស្វករកុំព្យូទ័រអាចប្រើប្រាស់នូវគំរូនៃដំណោះស្រាយប្រសើរបំផុតដែលធ្លាប់ប្រើពីមុនមក អនុវត្តចំពោះបញ្ហាថ្មីៗផ្សេងទៀតដោយពុំចាំបាច់បង្កើតដំណោះស្រាយថ្មីទេ។ វិស្វករដែលមានពិសោធន៍រឿងនេះអាចដាក់ចេញនូវដំណោះស្រាយយ៉ាងងាយស្រួល ហើយពួកគេក៏មានសមត្ថភាពយល់ការសរសេរកូដរបស់អ្នកដទៃបានយ៉ាងឆាប់រហ័សផងដែរ។

គំរូនៃការរៀបចំកូដតាមប្រភេទ និងការប្រើប្រាស់របស់វា

គំរូនៃការរៀបចំកូដចែកជាប្រភេទគឺ ប្រភេទគំរូបែបបង្កើត (Creational Pattern) ប្រភេទគំរូបែបចាត់ចែង (Structural Pattern) និងប្រភេទគំរូបែបប្រតិបត្តិ (Behavioral Pattern) ដែលការប្រើប្រាស់របស់វាត្រូវបានសង្ខេបដូចខាងក្រោម៖

១. ប្រភេទគំរូបែបបង្កើត (Creational Pattern)

គំរូបែបឯកមជ្ឈិម (Singleton Pattern)

បង្កើត Object មួយអាចប្រើប្រាស់ជាសាកល។

គំរូបែបអត្តនិមិត្ត (Prototype Pattern)

បង្កើត Object មួយដោយចម្លងចេញពី Object មួយទៀត។

គំរូបែបផលិតកម្ម (Factory Pattern)

ប្រមូលផ្តុំការបង្កើត Object នៃប្រភេទប្រហាក់ប្រហែលគ្នា កុំឱ្យរំពឹងប៉ាន់ប៉ាយ។

គំរូបែបស្ថាបនិក (Builder Pattern)

បង្កើត Object ស្មុគស្មាញមួយដោយស្ថាបនាផ្នែកនានារបស់វាដាច់ដោយឡែកពីគ្នា។

២. ប្រភេទគំរូបែបចាត់ចែង (Structural Pattern)

គំរូបែបផ្សិតម្រូវ (Adapter Pattern)

ធ្វើឱ្យ Object ពីរ ឬច្រើនអាចទាក់ទងគ្នាបានទោះបីសន្លានកម្ម (Interface) ផ្សេងគ្នា។

គំរូបែបអន្តរទំនាក់ទំនង (Bridge Pattern)

ផ្តាច់ Abstraction ចេញពី Implementation

គំរូបែបបន្ថែមលម្អ (Decorator Pattern)

បន្ថែម ឬលុប Responsibility នៃ Object ពេលដំណើរការ។

គំរូបែបសមាសចម្រុះ (Composite Pattern)

អនុញ្ញាតឱ្យ Client ប្រើប្រាស់ Object ឬក្រុមនៃ Object ដូចជា Object ប្រភេទតែមួយ។

៣. ប្រភេទគំរូមេឃប្រតិបត្តិ (Behavioral Pattern)

គំរូបែបវដ្តដំណើរ (Iterator Pattern)

Access ទៅលើសំណុំនៃ Object ដោយប្រយោល។

គំរូបែបស្វ័យគ្រប់គ្រង (Command Pattern)

វិចខ្ជប់សំណូមពរ (Request) ក្នុង Object និងរក្សាទុកសម្រាប់ដំណើរការនៅពេលក្រោយ។

គំរូបែបរក្សាចំណាំ (Memento Pattern)

បង្កើត Object ជា Read-Only ដើម្បីផ្ទុកទិន្នន័យនៃ Object ដើមប្រភេទណាមួយ។

គំរូបែបសណ្ឋានលក្ខខណ្ឌ (State Pattern)

អនុញ្ញាតឱ្យ Object ប្តូរនូវអាកប្បកិរិយា និងមុខងាររបស់ខ្លួនអាស្រ័យទៅលើតម្លៃផ្ទៃក្នុងរបស់វា។

គំរូបែបឧត្តតុល្យ (Template Method Pattern)

កំណត់នូវគ្រោងឆ្លឹង និងពុម្ពនៃ Class ដើម សម្រាប់ការបន្តមរតកប្រើប្រាស់ទៅ Class ជំនាន់ក្រោយ។

គំរូបែបយុទ្ធសាស្ត្រ (Strategy Pattern)

វិចខ្ជប់ Algorithm នៅក្នុងយុទ្ធសាស្ត្រផ្សេងៗគ្នា ដើម្បីប្រើប្រាស់សម្រេចគោលបំណងជាក់លាក់។

គំរូបែបសង្កេតការណ៍ (Observer Pattern)

អនុញ្ញាតឱ្យកំណែប្រែនៃសណ្ឋាន ឬ លក្ខណៈ របស់ Class ដើម អាចបញ្ជូនទៅកាន់ Class ជំនាន់ក្រោយបាន។

គំរូបែបទេសចរណ៍ (Visitor Pattern)

កំណត់នូវប្រតិបត្តិការថ្មីដោយមិនប៉ះពាល់ដល់ Class ជាតួអង្គផ្ទុកនូវលក្ខណៈទាំងឡាយ។

គំរូបែបបំណកប្រែ (Interpreter Pattern)

បំប្លែងទិន្នន័យពីទម្រង់មួយទៅទម្រង់ផ្សេងៗទៀត។

គំរូបែបសង្វាក់ទំនួលខុសត្រូវ (Chain of Responsibility Pattern)

ប្រគល់ការទំនួលខុសត្រូវទៅ Class ម្តងមួយៗ នៅក្នុងខ្សែសង្វាក់នៃការងារ ដើម្បីប្រតិបត្តិ (Process)។

គំរូបែបសន្លាតកម្ម (Model View Controller Pattern)

ផ្តាច់ចេញពីគ្នារវាង Model (Business Logic) និង View (Interface) ដោយមាន Controller ជាអ្នកសម្របសម្រួល (សន្លាតការ)។

ប្រភេទគំរូបែបបង្កើត

(Creational Pattern)

គំរូបែបឯកមជ្ឈិម

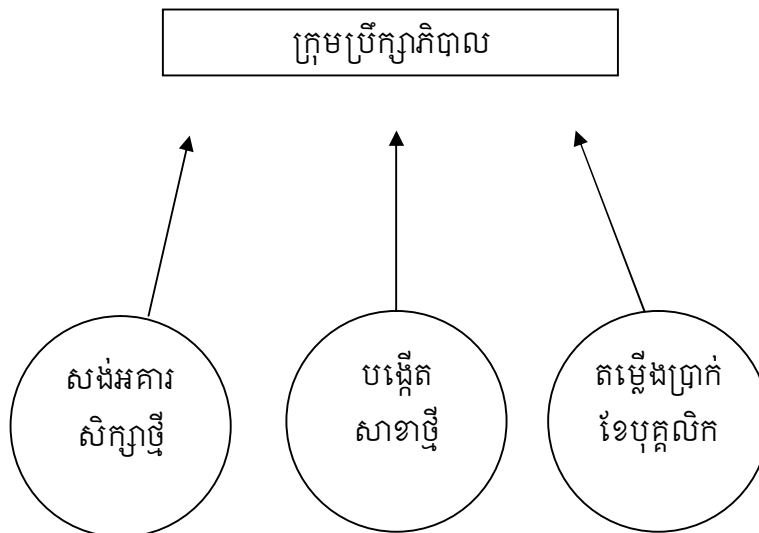
(Singleton Pattern)

គោលដៅ

បង្កើត Object មួយអាចប្រើប្រាស់ជាសាកល។

ជីវិតពិត

បង្កើតក្រុម ប្រឹក្សាភិបាលមួយសម្រាប់គ្រប់គ្រងសាកលវិទ្យាល័យមួយ។

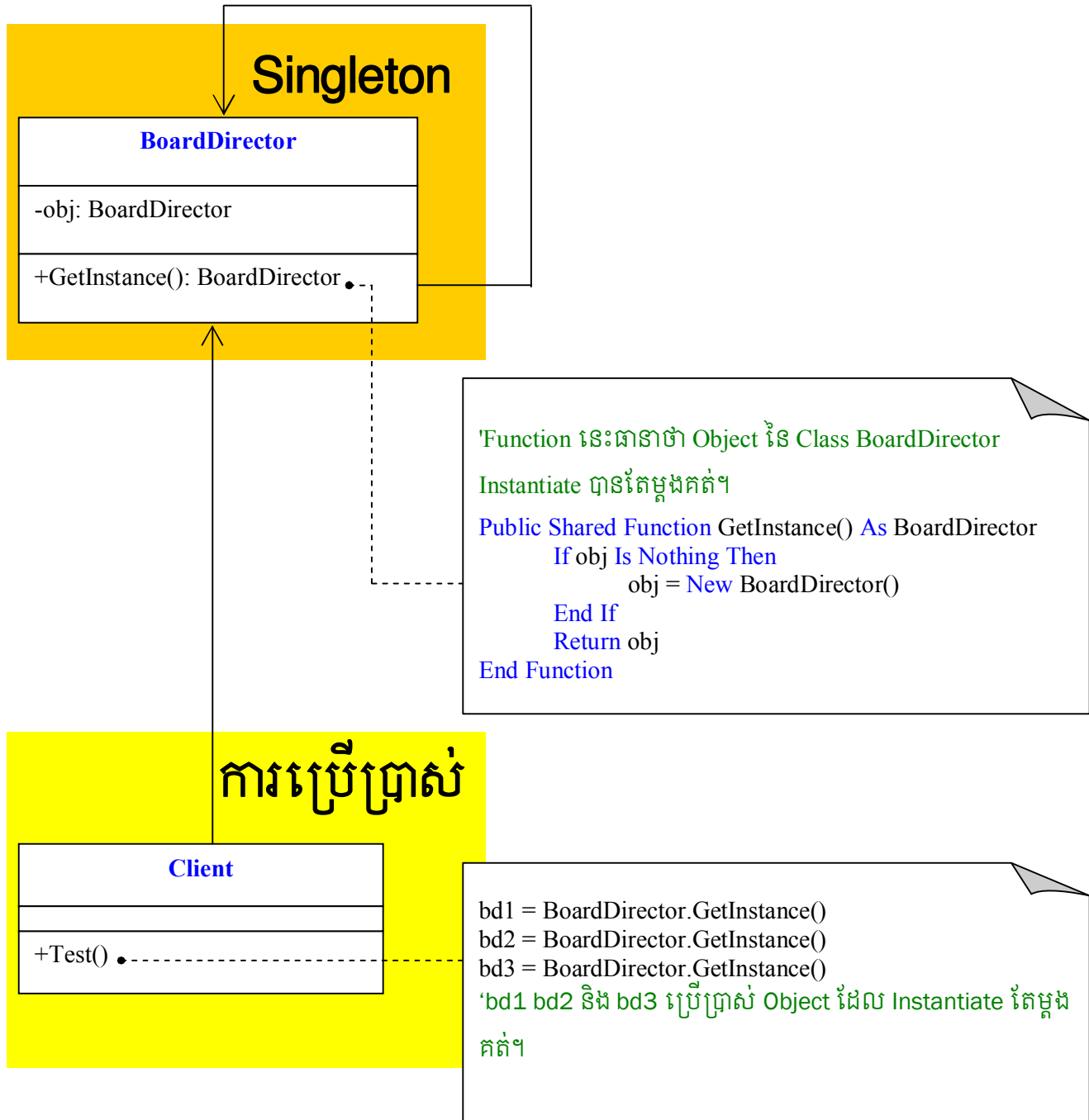


ការងារប្រមូលផ្តុំលើក្រុមប្រឹក្សាភិបាលដូច្នេះ នាំឱ្យពួកគាត់អាចក្តាប់ការងារបានទាំងអស់ ហើយអាចឱ្យគាត់ដើរតួបញ្ជូនព័ត៌មានរវាងផ្នែកផ្សេងៗនៅក្នុងសាកលវិទ្យាល័យ។

ដូច្នេះ

ការបង្កើតនូវ Object អាចប្រើប្រាស់ជាសាកលក៏ដើម្បីអាចក្តាប់បាននូវព័ត៌មាន និងបញ្ជូនព័ត៌មានរវាងផ្នែកផ្សេងៗ នៅក្នុង Project យើងផងដែរ។

ជំពូក UML នៃការបែងចែកឯកតា



ការអនុវត្ត

Module Singleton

'Singleton

'Class BoardDirector (ក្រុមប្រឹក្សាភិបាល) គឺជា Class ឯកមជ្ឈិម។

Public Class BoardDirector

'Keyword Shared មានន័យថាវាប្រើប្រាស់បានសម្រាប់ Class ទាំងមូល ហើយពុំចាំបាច់ Instantiate ឡើយ។

Private Shared obj As BoardDirector

Private Sub New()

Console.WriteLine("I was instantiated only once.")

End Sub

'Function នេះធានាថា Object នៃ Class BoardDirector Instantiate បានតែម្តងគត់។

Public Shared Function GetInstance() As BoardDirector

If obj Is Nothing Then

obj = New BoardDirector()

End If

Return obj

End Function

'សម្រាប់អ្នកពាក់ព័ន្ធ (Client) ទំនាក់ទំនងមក Class BoardDirector។

Public Function GetApproval(ByVal request As String) As String

Return request & " approved!"

End Function

End Class ' BoardDirector

'Class ដែលប្រើប្រាស់ Class BoardDirector

Public Class Client

Public Shared Sub Test()

Dim bd1 As BoardDirector

'Object នៃ Class BoardDirector ត្រូវបាន Instantiate ថ្មី។

bd1 = BoardDirector.GetInstance()

Console.WriteLine(bd1.GetApproval("build a new building"))

Dim bd2 As BoardDirector

'Object នៃ Class BoardDirector ពុំបាន Instantiate ថ្មីទេ

'គឺវាប្រើប្រាស់ Object ចាស់ដដែល។

bd2 = BoardDirector.GetInstance()

Console.WriteLine(bd2.GetApproval("open a new branch"))

Dim bd3 As BoardDirector

'Object នៃ Class BoardDirector ពុំបាន Instantiate ថ្មីទេ

'គឺវាប្រើប្រាស់ Object ចាស់ដដែល។

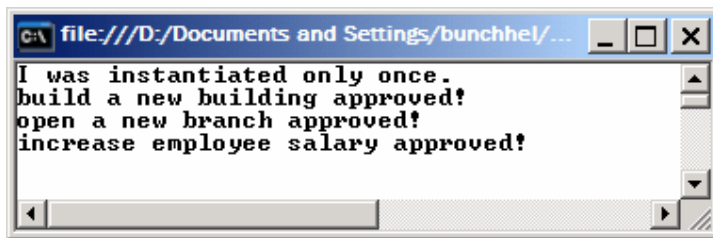
bd3 = BoardDirector.GetInstance()

Console.WriteLine(bd3.GetApproval("increase employee salary"))

```
        Console.Read()  
    End Sub  
End Class 'Client  
  
Sub Main()  
    Client.Test()  
End Sub
```

End Module

លទ្ធផល



គំរូបែបអត្តនិមិត្ត

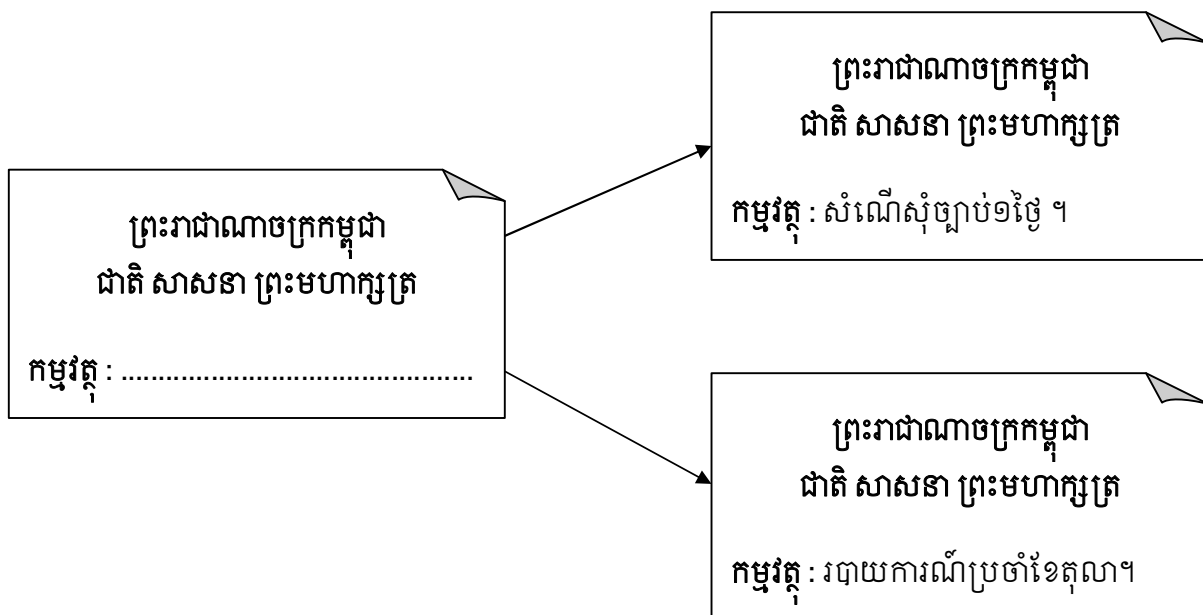
(Prototype Pattern)

គោលដៅ

បង្កើត Object មួយដោយចម្លងចេញពី Object មួយទៀត។

ជីវិតពិត

ចម្លងអត្ថបទពុម្ពគំរូ ដើម្បីកែសម្រួលបង្កើតទៅជាអត្ថបទថ្មី។

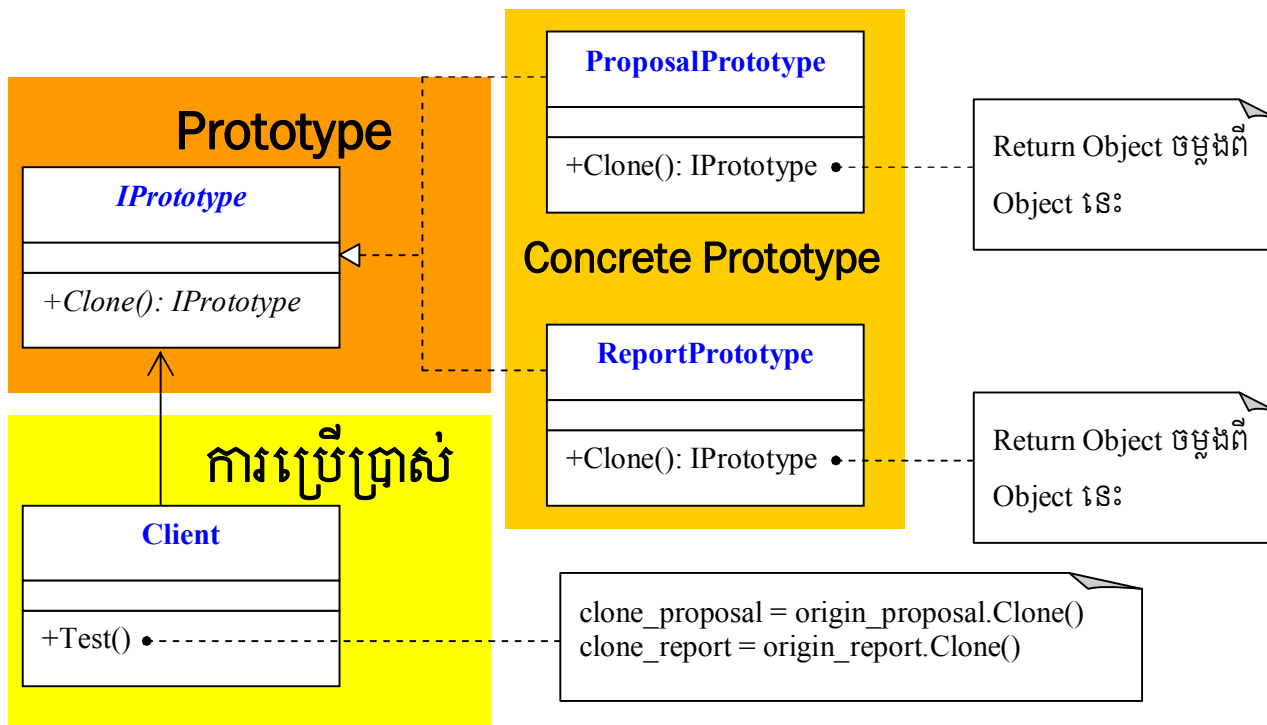


ការចម្លងអត្ថបទពុម្ពគំរូដូច្នេះវាទាំងឡាយចំណេញពេលវេលា ដោយយើងគ្រាន់តែកែបន្ថែមបន្ថយបន្តិច
បន្តួចប៉ុណ្ណោះ យើងនឹងទទួលបាននូវអត្ថបទថ្មីចង់បាន។

ដូច្នេះ

ការបង្កើតនូវ Object មួយតាមរយៈការចម្លងយ៉ាងដូច្នេះវាធ្វើឱ្យយើងចំណេញពេលក្នុងការសរសេរ
កូដ។

ជ្រាប UML នៃគំរូបបង្កនិមិត្ត



ការអនុវត្ត

Module Prototype

'Prototype

Public Interface IPrototype

Function Clone() As IPrototype

End Interface

'Concrete Prototype

Public Class ProposalPrototype

Implements IPrototype

Private what_country As String = "<enter country name>"

Private what_subject As String = "<enter proposal subject>"

Public Property Country() As String

Get

Return what_country

End Get

Set(ByVal value As String)

what_country = value

End Set

End Property

Public Property Subject() As String

Get

Return what_subject

End Get

Set(ByVal value As String)

what_subject = value

End Set

End Property

'ចម្លងរាល់ Property របស់ Object ចាស់ទៅ Object ថ្មី ហើយ Return Object ថ្មីនោះ

Public Function Clone() As IPrototype Implements IPrototype.Clone

Dim clone_obj As ProposalPrototype

clone_obj = New ProposalPrototype()

clone_obj.Country = what_country

clone_obj.Subject = what_subject

Return clone_obj

End Function

Public Sub Display()

Console.WriteLine(what_country)

Console.WriteLine("Proposed for: " & what_subject)

Console.WriteLine()

End Sub

End Class

'Concrete Prototype

Public Class ReportPrototype

Implements IPrototype

Private what_country As String = "<enter country name>"

```
Private what_subject As String = "<enter report subject>"
```

```
Public Property Country() As String
    Get
        Return what_country
    End Get
    Set(ByVal value As String)
        what_country = value
    End Set
End Property
```

```
Public Property Subject() As String
    Get
        Return what_subject
    End Get
    Set(ByVal value As String)
        what_subject = value
    End Set
End Property
```

‘ចម្លងរាល់ Property របស់ Object ចាស់ទៅ Object ថ្មី ហើយ Return Object ថ្មីនោះ

```
Public Function Clone() As IPrototype Implements IPrototype.Clone
    Dim clone_obj As ReportPrototype
    clone_obj = New ReportPrototype()
    clone_obj.Country = what_country
    clone_obj.Subject = what_subject
    Return clone_obj
End Function
```

```
Public Sub Display()
    Console.WriteLine(what_country)
    Console.WriteLine("Report on: " & what_subject)
    Console.WriteLine()
End Sub
End Class
```

```
Public Class Client
    Public Shared Sub Test()
        ‘ប្រកាសគំរូនៃសំណើ និងបង្ហាញវា
        Dim origin_proposal As ProposalPrototype = New ProposalPrototype()
        origin_proposal.Display()
```

‘បញ្ចូលឈ្មោះប្រទេស និងកម្មវត្ថុរបស់គំរូនៃសំណើ ហើយបង្ហាញវា

```
origin_proposal.Country = "Kingdom of Cambodia"
origin_proposal.Subject = "Take Leave"
origin_proposal.Display()
```

‘ចម្លងគំរូនៃសំណើទៅ Object មួយទៀត រួចធ្វើការកែសម្រួលតែលើកម្មវត្ថុ
‘ដោយពុំចាំបាច់បំពេញឈ្មោះប្រទេសឡើងវិញទេ រួចបង្ហាញ Object ថ្មីនោះ

```
Dim clone_proposal As ProposalPrototype = origin_proposal.Clone()
clone_proposal.Subject = "1.000.000R"
clone_proposal.Display()
```

ប្រកាសគំរូនៃរបាយការណ៍ និងបង្ហាញវា

```
Dim origin_report As ReportPrototype = New ReportPrototype()  
origin_report.Display()
```

បញ្ចូលឈ្មោះប្រទេស និងកម្មវត្ថុរបស់គំរូនៃរបាយការណ៍ និងបង្ហាញវា

```
origin_report.Country = "Kingdom of Cambodia"  
origin_report.Subject = "October Progressive Report"  
origin_report.Display()
```

ចម្លងគំរូនៃរបាយការណ៍ទៅ Object មួយទៀត រួចធ្វើការកែសម្រួលតែលើកម្មវត្ថុ

ដោយពុំចាំបាច់បំពេញឈ្មោះប្រទេសឡើងវិញទេ រួចបង្ហាញ Object ថ្មីនោះ

```
Dim clone_report As ReportPrototype = origin_report.Clone()  
clone_report.Subject = "November Progressive Report"  
clone_report.Display()
```

```
Console.Read()
```

```
End Sub
```

```
End Class 'Client
```

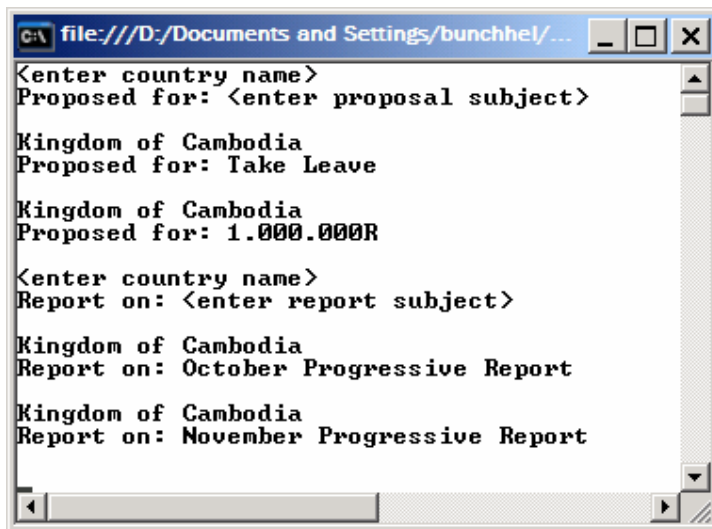
```
Sub Main()
```

```
Client.Test()
```

```
End Sub
```

```
End Module
```

លទ្ធផល



គំរូបែបផលិតកម្ម

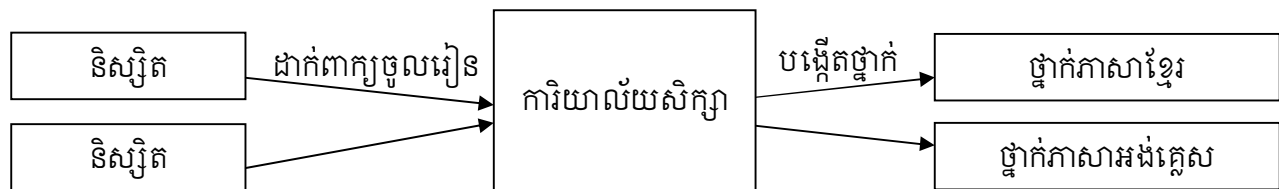
(Factory Pattern)

គោលដៅ

ប្រមូលផ្តុំការបង្កើត Object នៃប្រភេទប្រហាក់ប្រហែលគ្នា កុំឱ្យប៉ាន់ប៉ាយ។

ជីវិតពិត

ការិយាល័យសិក្សារៀបចំថ្នាក់ទៅតាមមុខវិជ្ជាផ្សេងៗដែលនិស្សិតដាក់ពាក្យចង់រៀន។

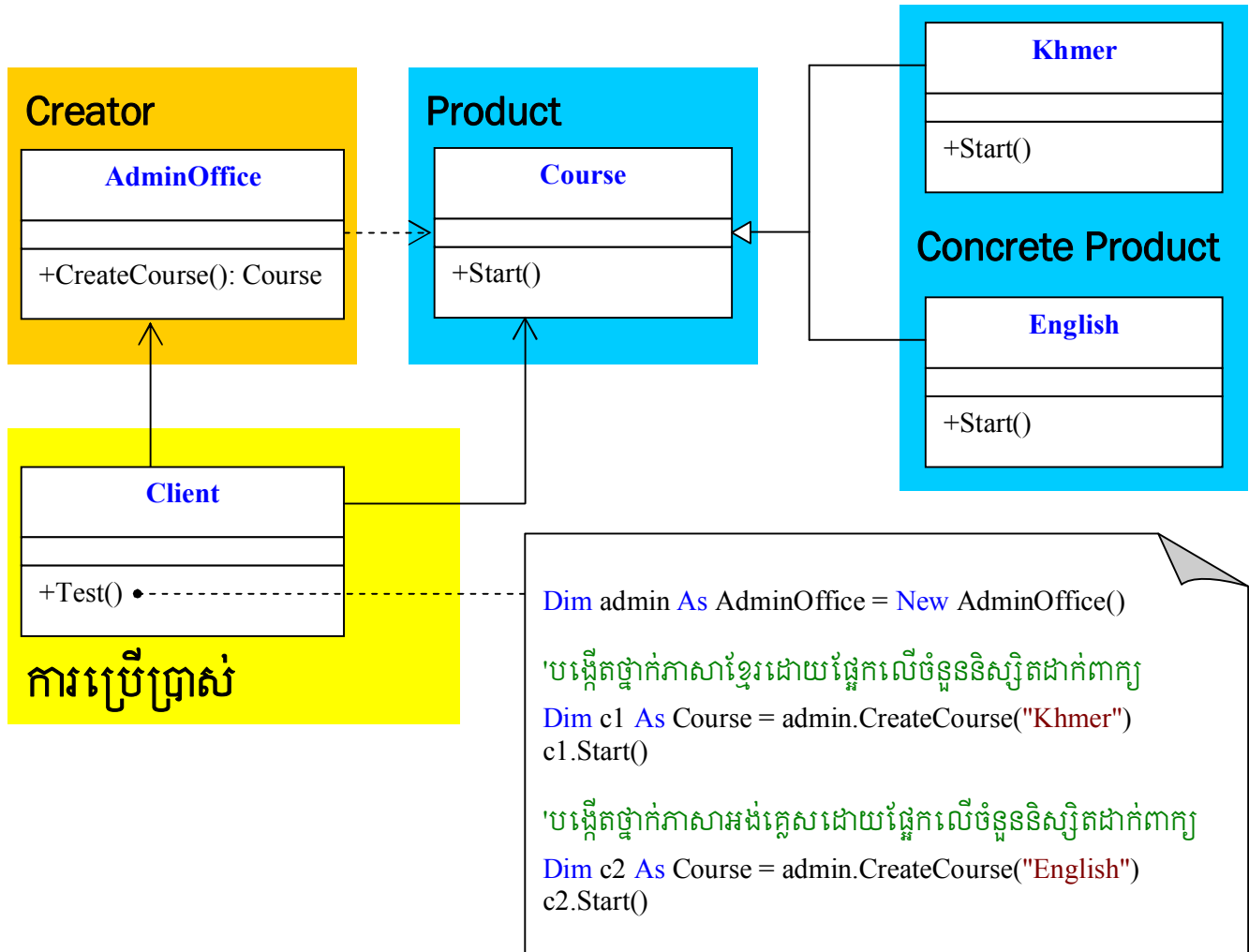


រាល់ការកែប្រែនៃមុខវិជ្ជា ឬម៉ោងសិក្សារបស់និស្សិត ក៏ដូចជាការផ្លាស់ប្តូរទីតាំងនៃថ្នាក់រៀន ឬគ្រូបង្រៀន ត្រូវធ្វើឡើងតាមរយៈការិយាល័យសិក្សា ដើម្បីផ្តល់ភាពងាយស្រួលក្នុងការផ្លាស់ប្តូរ។

ជូន្នេះ

ប្រមូលផ្តុំការបង្កើត Object នៃប្រភេទប្រហាក់ប្រហែលគ្នា កុំឱ្យប៉ាន់ប៉ាយ ក៏ដើម្បីងាយស្រួលដល់ការកែប្រែដែរ ដោយសារវានៅមួយកន្លែងនាំឱ្យងាយរក ។

រូបភាព UML នៃគំរូបែបផលិតកម្ម



ការអនុវត្ត

Module Factory

```
'Product
Public Class Course
    Overridable Sub Start()
    End Sub
End Class 'End Class Course

'Concrete Product
Public Class Khmer
    Inherits Course

    Public Overrides Sub Start()
        Console.WriteLine("Khmer Course Started.")
    End Sub
End Class 'End Class Khmer

'Concrete Product
Public Class English
    Inherits Course

    Public Overrides Sub Start()
        Console.WriteLine("English Course Started.")
    End Sub
End Class 'End Class English

'Creator
Public Class AdminOffice
    Public Function CreateCourse(ByVal what_course As String) As Course
        Select Case what_course
            Case "Khmer"
                Return New Khmer()
            Case "English"
                Return New English()
        End Select
    End Function
End Class

Public Class Client
    Public Shared Sub Test()
        Dim admin As AdminOffice = New AdminOffice()

        'បង្កើតថ្នាក់ភាសាខ្មែរដោយផ្អែកលើចំនួននិស្សិតដាក់ពាក្យ
        Dim c1 As Course = admin.CreateCourse("Khmer")
        c1.Start()

        'បង្កើតថ្នាក់ភាសាអង់គ្លេសដោយផ្អែកលើចំនួននិស្សិតដាក់ពាក្យ
        Dim c2 As Course = admin.CreateCourse("English")
        c2.Start()

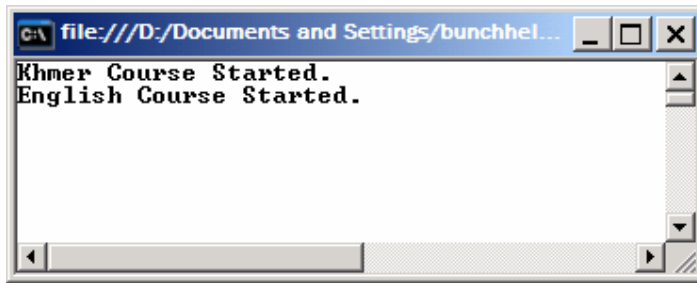
        Console.Read()
    End Sub
```

End Class

```
Sub Main()  
    Client.Test()  
End Sub
```

End Module

លទ្ធផល



គំរូបែបស្ថាបនិក

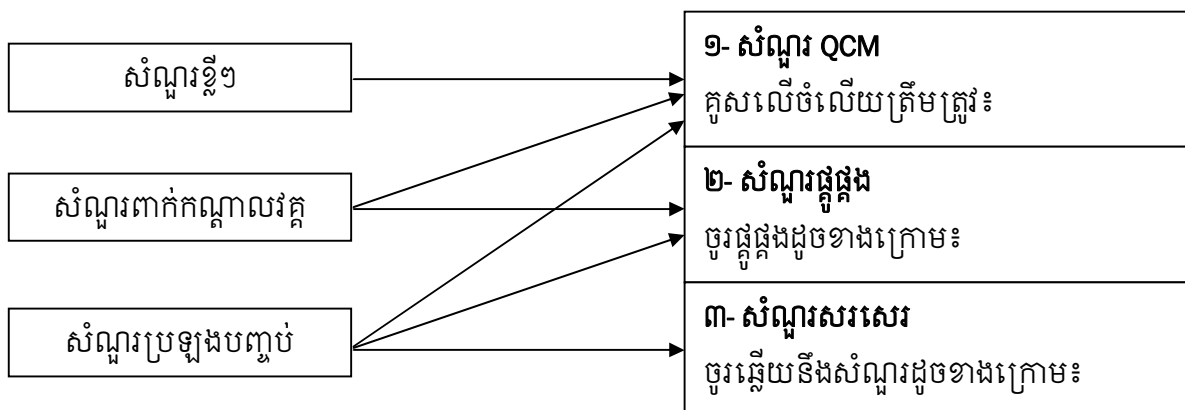
(Builder Pattern)

គោលដៅ

បង្កើត Object ស្មុគស្មាញមួយដោយស្ថាបនាផ្នែកនានាបស់វាដាច់ដោយឡែកពីគ្នា។

ជីវិតពិត

បង្កើតវិញ្ញាសាប្រឡង ដោយបែងចែកផ្នែកនីមួយៗផ្សេងៗពីគ្នា ។

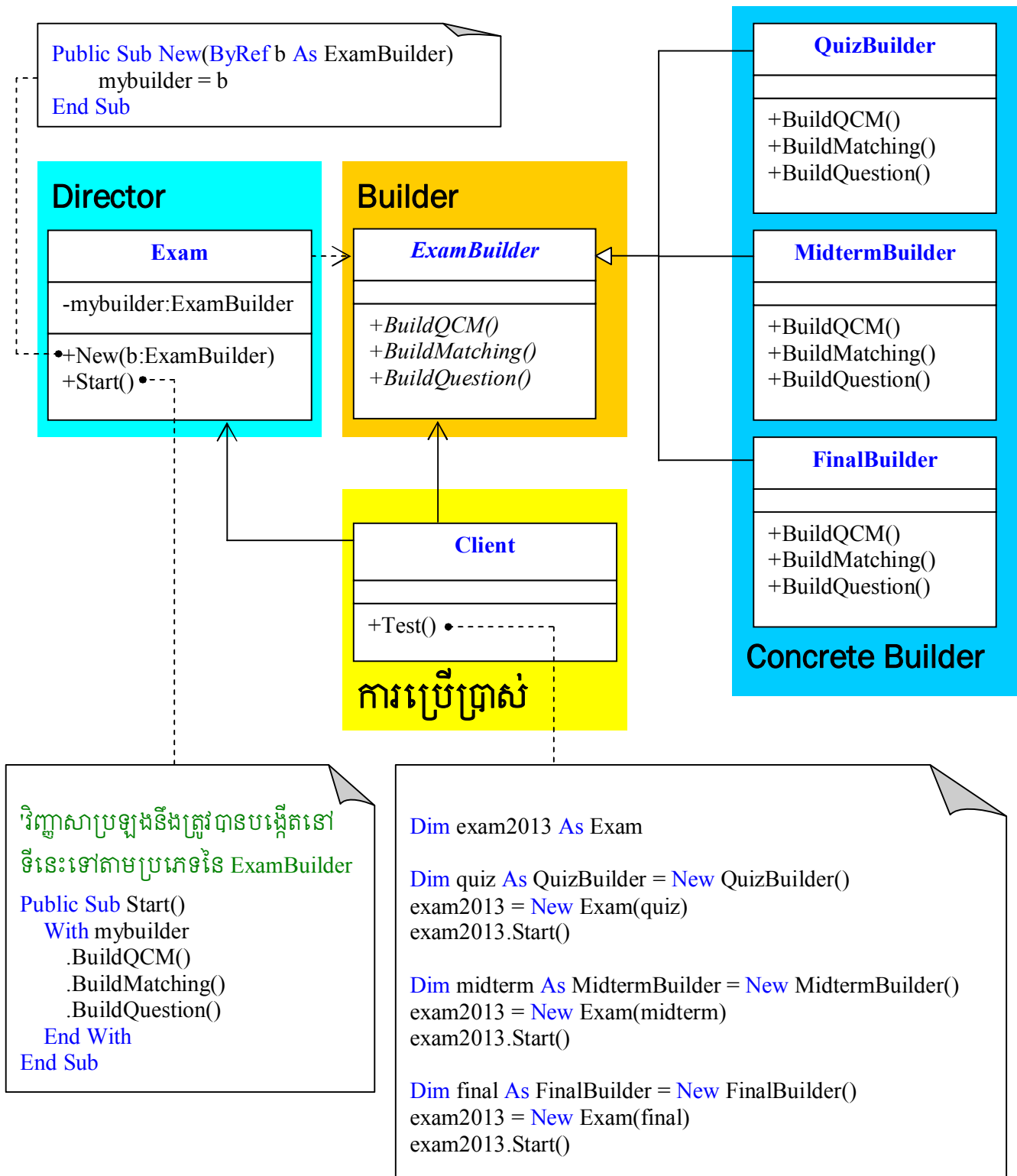


ការចែកយ៉ាងដូច្នេះអាចធ្វើឱ្យការបង្កើតមានភាពឆាប់រហ័ស មានភាពងាយស្រួល និងងាយបត់បែន ក្នុងការចេញវិញ្ញាសា។

ដូច្នេះ

ការបង្កើត Object ស្មុគស្មាញមួយ អាចមានភាពឆាប់រហ័សដោយការចែកការងារទៅ Thread ដាច់ដោយឡែកពីគ្នា ហើយការបំបែកវាឱ្យទៅជាការងារតូចៗក៏ធ្វើឱ្យងាយស្រួលដល់ការបង្កើត Object ជាមួយភាពបត់បែនផង។

ផ្សារក្រាម UML នៃគំរូបែបស្ថាបនិក



ការអនុវត្ត

Module Builder

'Director

Public Class Exam

Private mybuilder As ExamBuilder

Public Sub New(ByRef b As ExamBuilder)

mybuilder = b

End Sub

វិញ្ញាសាប្រឡងនឹងត្រូវបានបង្កើតនៅទីនេះទៅតាមប្រភេទនៃ ExamBuilder

Public Sub Start()

With mybuilder

.BuildQCM()

.BuildMatching()

.BuildQuestion()

End With

Console.WriteLine()

End Sub

End Class

'Abstract Builder

MustInherit Class ExamBuilder

Public MustOverride Sub BuildQCM()

Public MustOverride Sub BuildMatching()

Public MustOverride Sub BuildQuestion()

End Class

'Concrete Builder

Public Class QuizBuilder

Inherits ExamBuilder

Public Overrides Sub BuildQCM()

Console.WriteLine("QCM")

End Sub

Public Overrides Sub BuildMatching()

'គ្មានសំណួរផ្គុំផ្គងទេសម្រាប់ Quiz

End Sub

Public Overrides Sub BuildQuestion()

'គ្មានសំណួរសរសេរទេសម្រាប់ Quiz

End Sub

End Class

'Concrete Builder

Public Class MidtermBuilder

Inherits ExamBuilder

Public Overrides Sub BuildQCM()

Console.WriteLine("QCM")

End Sub

```
Public Overrides Sub BuildMatching()
    Console.WriteLine("Matching")
End Sub
```

```
Public Overrides Sub BuildQuestion()
    'គ្មានសំណួរសរសេរទេសម្រាប់ Midterm
End Sub
End Class
```

'Concrete Builder

```
Public Class FinalBuilder
    Inherits ExamBuilder
```

```
Public Overrides Sub BuildQCM()
    Console.WriteLine("QCM")
End Sub
```

```
Public Overrides Sub BuildMatching()
    Console.WriteLine("Matching")
End Sub
```

```
Public Overrides Sub BuildQuestion()
    Console.WriteLine("Question")
End Sub
End Class
```

'ការប្រើប្រាស់

```
Public Class Client
    Public Shared Sub Test()
        Dim exam2013 As Exam
```

```
        Console.WriteLine("What we have for quiz:")
        Dim quiz As QuizBuilder = New QuizBuilder()
        exam2013 = New Exam(quiz)
        exam2013.Start()
```

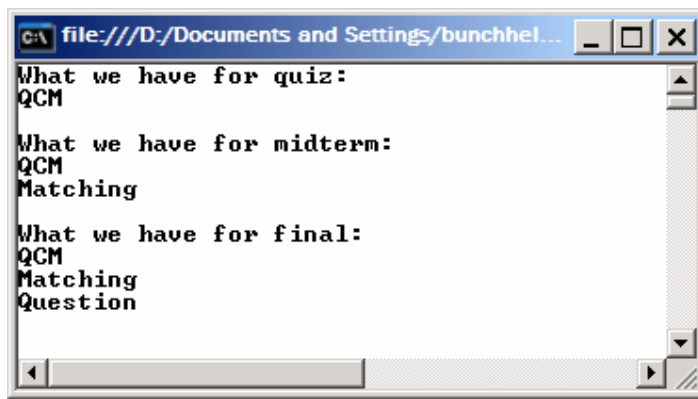
```
        Console.WriteLine("What we have for midterm:")
        Dim midterm As MidtermBuilder = New MidtermBuilder()
        exam2013 = New Exam(midterm)
        exam2013.Start()
```

```
        Console.WriteLine("What we have for final:")
        Dim final As FinalBuilder = New FinalBuilder()
        exam2013 = New Exam(final)
        exam2013.Start()
```

```
        Console.Read()
    End Sub
End Class
```

```
Sub Main()
    Client.Test()
End Sub
End Module
```

លទ្ធផល



ប្រភេទគំរូបែបចាត់ចែង

(Structural Pattern)

គំរូបែបផ្សិតប្រូត

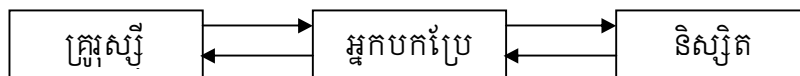
(Adapter Pattern)

គោលដៅ

ធ្វើឱ្យ Object ពីរ ឬច្រើនអាចទាក់ទងគ្នាបានទោះបីសន្លាកម្ម (Interface) ផ្សេងគ្នា។

ជីវិតពិត

គ្រូជនជាតិរុស្ស៊ីម្នាក់បង្រៀននិស្សិតខ្មែរដែលពុំចេះភាសារុស្ស៊ី ដូច្នេះដើម្បីអាចស្តាប់គ្នាបានត្រូវមានអ្នកបកប្រែភាសាម្នាក់។

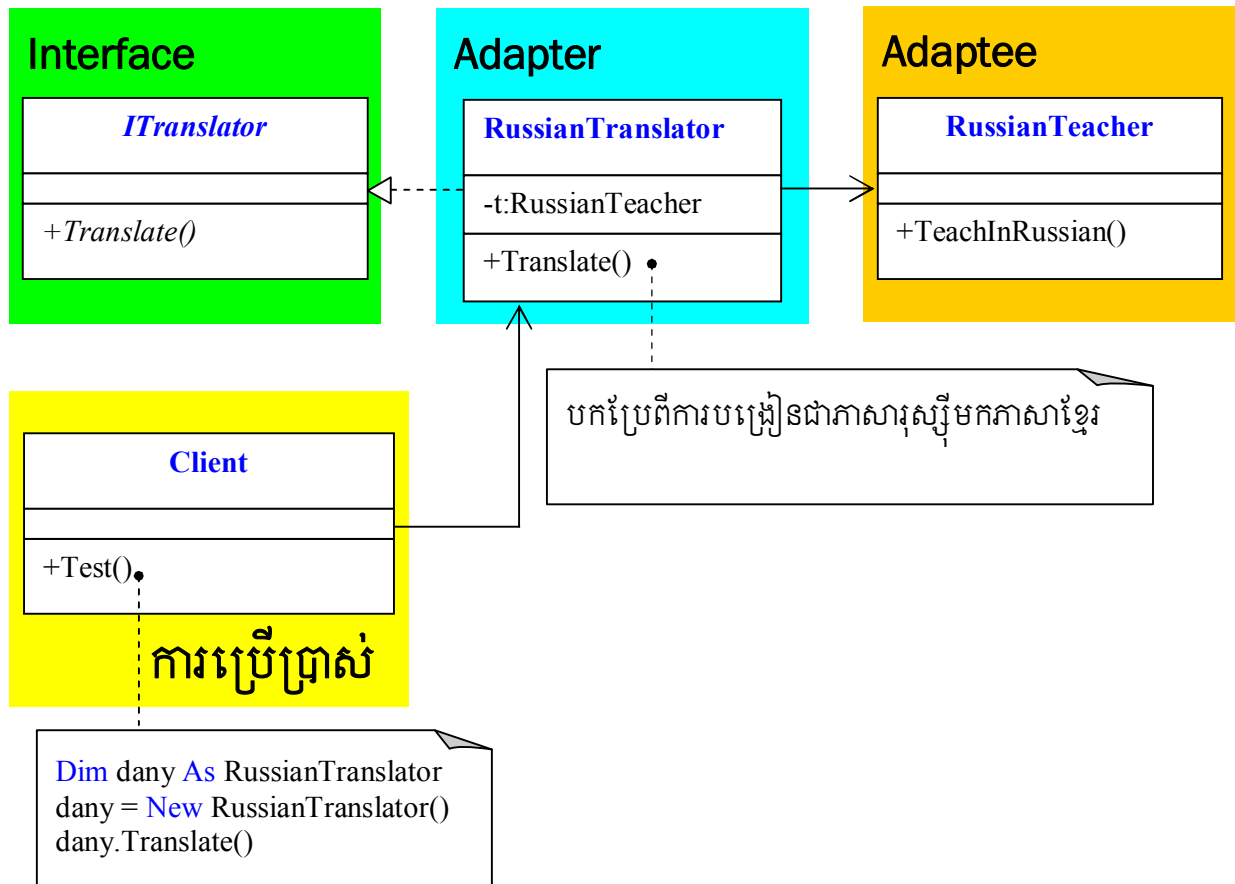


តាមមធ្យោបាយនេះ និស្សិតខ្មែរអាចទទួលបានការបណ្តុះបណ្តាលពីគ្រូគ្រប់ជនជាតិ ដោយយើងគ្រាន់តែរកអ្នកបកប្រែមកប៉ុណ្ណោះ។

ដូច្នេះ

Object ពីរ ឬច្រើនដែលមិនអាចទាក់ទងគ្នាដោយផ្ទាល់បានដោយសារសន្លាកម្ម (Interface) ផ្សេងគ្នា ក៏នៅតែអាចទាក់ទងគ្នាបានដោយសារការផ្សិតប្រូតពី Object ទី៣ (អ្នកបកប្រែ) នេះឯង។

ជួរក្រាម UML ដែលបង្ហាញពីការប្រើប្រាស់



ការអនុវត្ត

Module Adapter

```
'Interface
Public Interface ITranslator
    Sub Translate()
End Interface

'Adaptee
Public Class RussianTeacher
    Public Sub TeachInRussian()
        Console.WriteLine("Teaching in Russian.")
    End Sub
End Class

'Adapter
Public Class RussianTranslator
    Implements ITranslator
    Private t As New RussianTeacher

    Public Sub OriginalSpeech()
        t.TeachInRussian()
    End Sub

    Public Sub Translate() Implements ITranslator.Translate
        Console.WriteLine("Translating Russian To Khmer.")
    End Sub
End Class

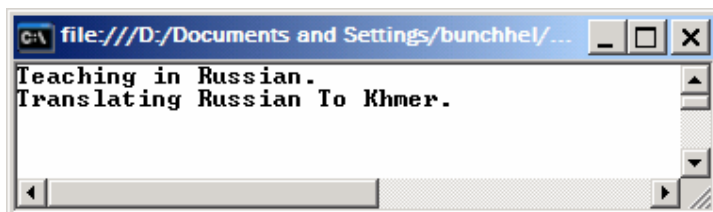
'This Class can be considered as students.
Public Class Client
    Public Shared Sub Test()
        Dim dany As RussianTranslator = New RussianTranslator()
        dany.OriginalSpeech()
        dany.Translate()

        Console.Read()
    End Sub
End Class

Sub Main()
    Client.Test()
End Sub

End Module
```

លទ្ធផល



គំរូបែបអណ្តូងទំនាក់ទំនង

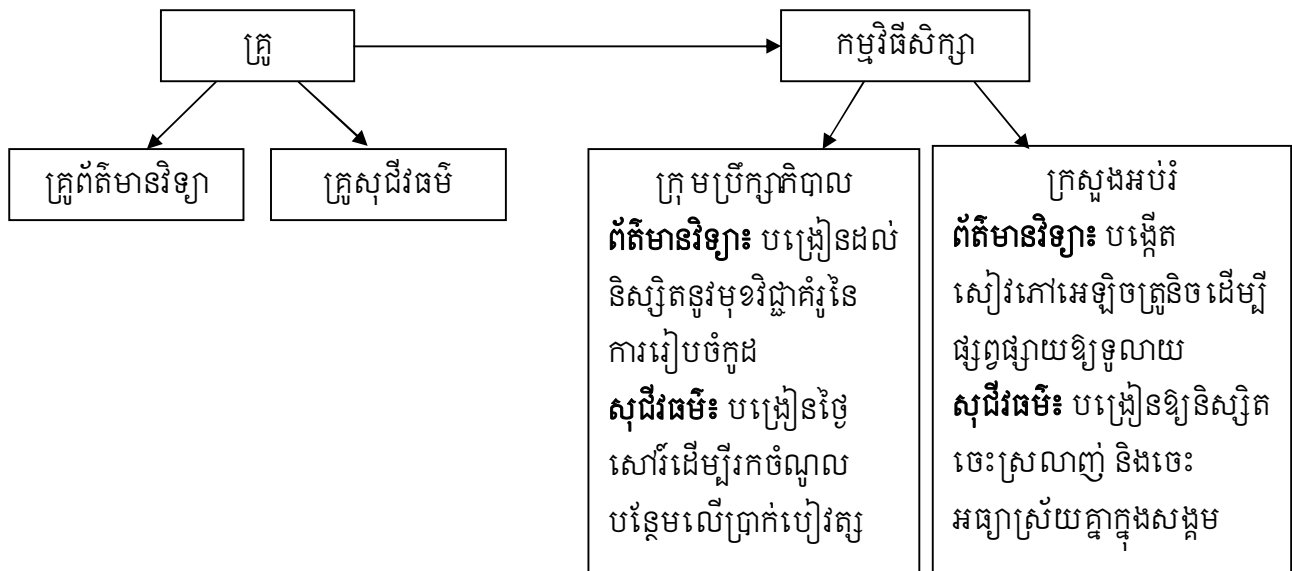
(Bridge Pattern)

គោលដៅ

ផ្តាច់ Abstraction ចេញពី Implementation

ជីវិតពិត

គ្រូត្រូវបង្រៀនទៅតាមកម្មវិធីសិក្សាដែលតម្រង់ទិសដោយក្រុម ប្រឹក្សាភិបាល និងក្រសួងអប់រំ។

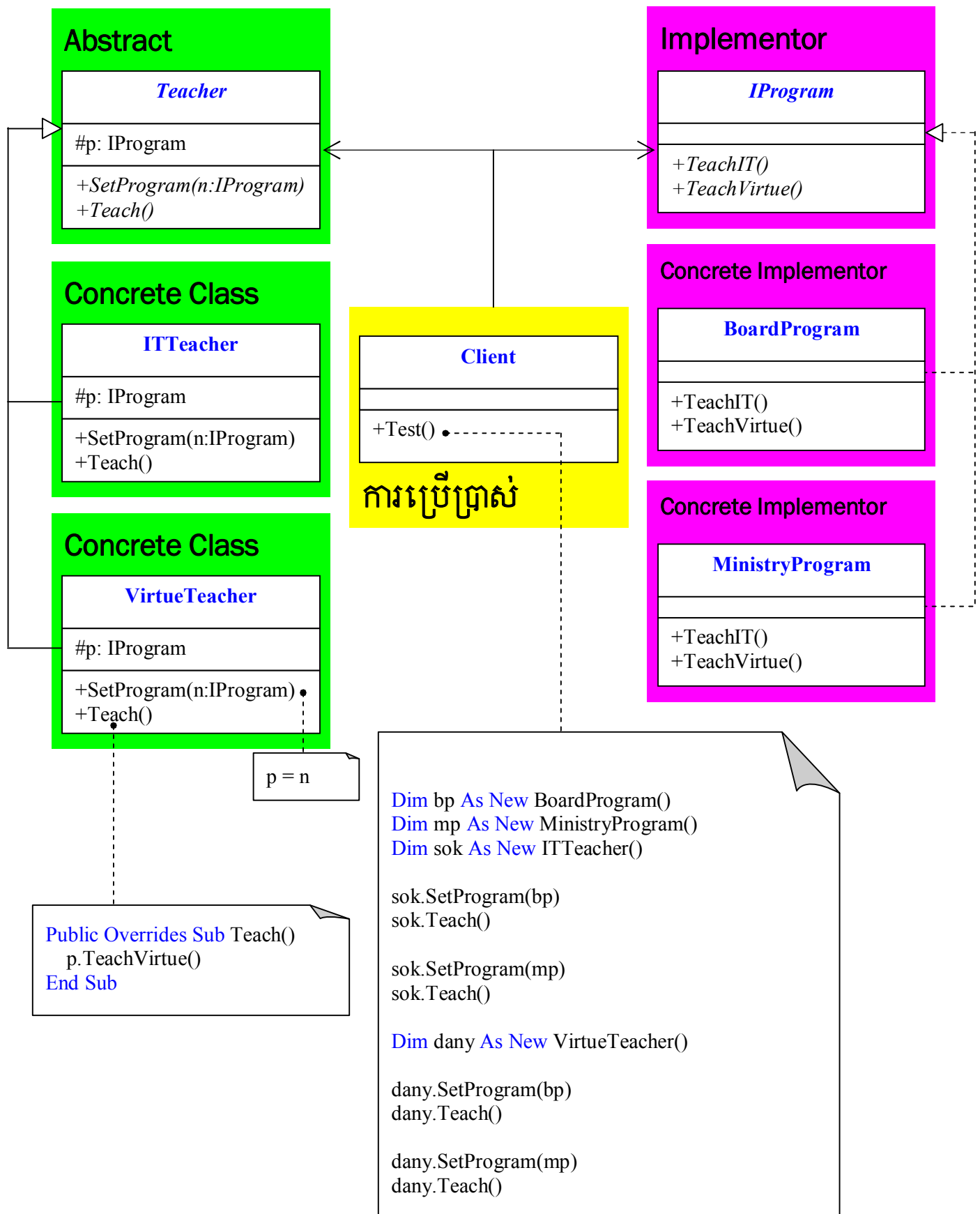


គ្រូអាចបត់បែនការបង្រៀនឱ្យស្របតាមកម្មវិធីសិក្សារបស់ក្រុមប្រឹក្សាភិបាល ឬក្រសួងអប់រំទៅតាម
កាលៈទេសៈសមស្រប ។

ដូច្នេះ

ការផ្តាច់ Abstraction ចេញពី Implementation ក៏ដើម្បីអាចឱ្យ Class ដែល Inherit ពី Abstraction
មានលទ្ធភាពជ្រើសរើស Implementation របស់ខ្លួនប្រកបទៅដោយភាពបត់បែន។

ដ្យាក្រាម UML នៃគំរូបអន្តរាគមន៍



ការអនុវត្ត

Module Bridge

'Implementor

```
Public Interface IProgram
    Sub TeachIT()
    Sub TeachVirtue()
End Interface
```

'Concrete Implementor

```
Public Class BoardProgram
    Implements IProgram
```

```
    Public Sub TeachIT() Implements IProgram.TeachIT
        Console.WriteLine("Board Program: Teach Design Pattern")
    End Sub
```

```
    Public Sub TeachVirtue() Implements IProgram.TeachVirtue
        Console.WriteLine("Board Program: Teach on Saturday too for better income.")
    End Sub
End Class
```

'Concrete Implementor

```
Public Class MinistryProgram
    Implements IProgram
```

```
    Public Sub TeachIT() Implements IProgram.TeachIT
        Console.WriteLine("Ministry Program: Encourage to establish ebook.")
    End Sub
```

```
    Public Sub TeachVirtue() Implements IProgram.TeachVirtue
        Console.WriteLine("Ministry Program: Teach students to love the society.")
    End Sub
End Class
```

'Abstract

```
Public Class Teacher
    Protected p As IProgram
```

'នៅទីនេះកម្មវិធីសិក្សាអាចត្រូវបានកំណត់ជាកម្មវិធីសិក្សារបស់ក្រុម មប្រឹក្សាភិបាល
'ឬរបស់ក្រសួងអប់រំ

```
    Overridable Sub SetProgram(ByRef n As IProgram)
        p = n
    End Sub
```

```
    Overridable Sub Teach()
    End Sub
End Class
```

'Concrete Class

```
Public Class ITTeacher
    Inherits Teacher
```

```

Public Overrides Sub Teach()
    p.TeachIT()
End Sub
End Class

```

'Concrete Class

```

Public Class VirtueTeacher
    Inherits Teacher

```

```

    Public Overrides Sub Teach()
        p.TeachVirtue()
    End Sub
End Class

```

```

Public Class Client

```

```

    Public Shared Sub Test()
        Dim bp As New BoardProgram()
        Dim mp As New MinistryProgram()
        Dim sok As New ITTeacher()
        Console.WriteLine("Sok as an IT Teacher:")

```

'សុខជាគ្រូព័ត៌មានវិទ្យា ត្រូវបានកំណត់ឱ្យបង្រៀនតាមកម្មវិធីសិក្សារបស់ក្រុមប្រឹក្សាភិបាល

```

sok.SetProgram(bp)
sok.Teach()

```

'សុខជាគ្រូព័ត៌មានវិទ្យា ត្រូវបានកំណត់ឱ្យបង្រៀនតាមកម្មវិធីសិក្សារបស់ក្រសួងអប់រំ

```

sok.SetProgram(mp)
sok.Teach()

```

```

Console.WriteLine("")

```

```

Dim dany As New VirtueTeacher()
Console.WriteLine("Dany as a Virtue Teacher:")

```

'ជានីជាគ្រូសុជីវធម៌ ត្រូវបានកំណត់ឱ្យបង្រៀនតាមកម្មវិធីសិក្សារបស់ក្រុមប្រឹក្សាភិបាល

```

dany.SetProgram(bp)
dany.Teach()

```

'ជានីជាគ្រូសុជីវធម៌ ត្រូវបានកំណត់ឱ្យបង្រៀនតាមកម្មវិធីសិក្សារបស់ក្រសួងអប់រំ

```

dany.SetProgram(mp)
dany.Teach()

```

```

Console.Read()

```

```

End Sub
End Class

```

```

Sub Main()
    Client.Test()
End Sub

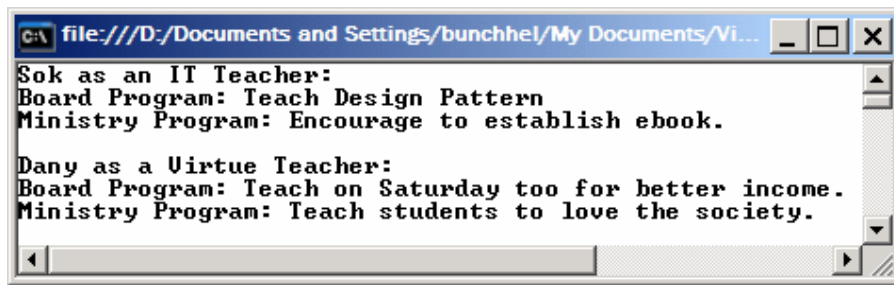
```

```

End Module

```

លទ្ធផល



គំរូបែបបទឆ្លែងលម្អ

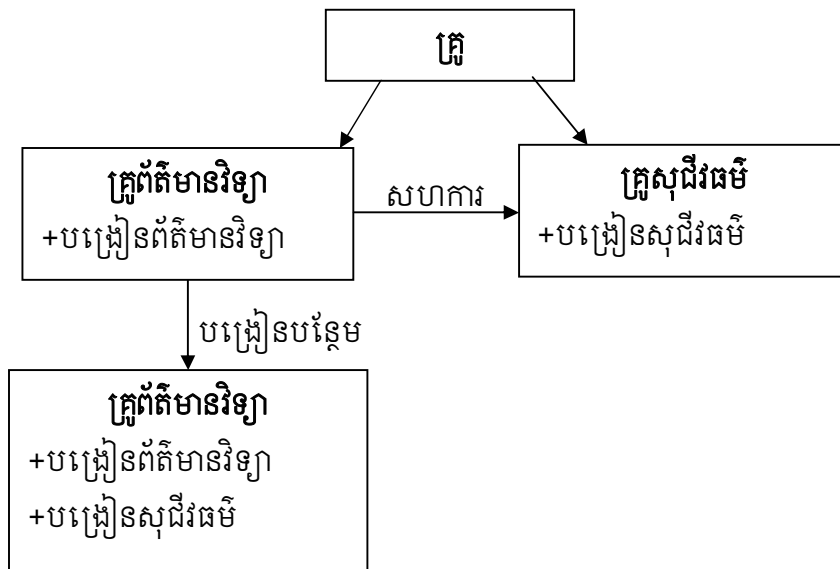
(Decorator Pattern)

គោលដៅ

បន្ថែម ឬ លុប Responsibility នៃ Object ពេលដំណើរការ។

ជីវិតពិត

គ្រូបង្រៀនមុខវិជ្ជាមួយត្រូវបានបន្ថែមកិច្ចការបង្រៀនផ្សេងទៀត ហើយអាចដកចេញនូវកិច្ចការបង្រៀននោះវិញក្រោយរយៈពេលណាមួយ។

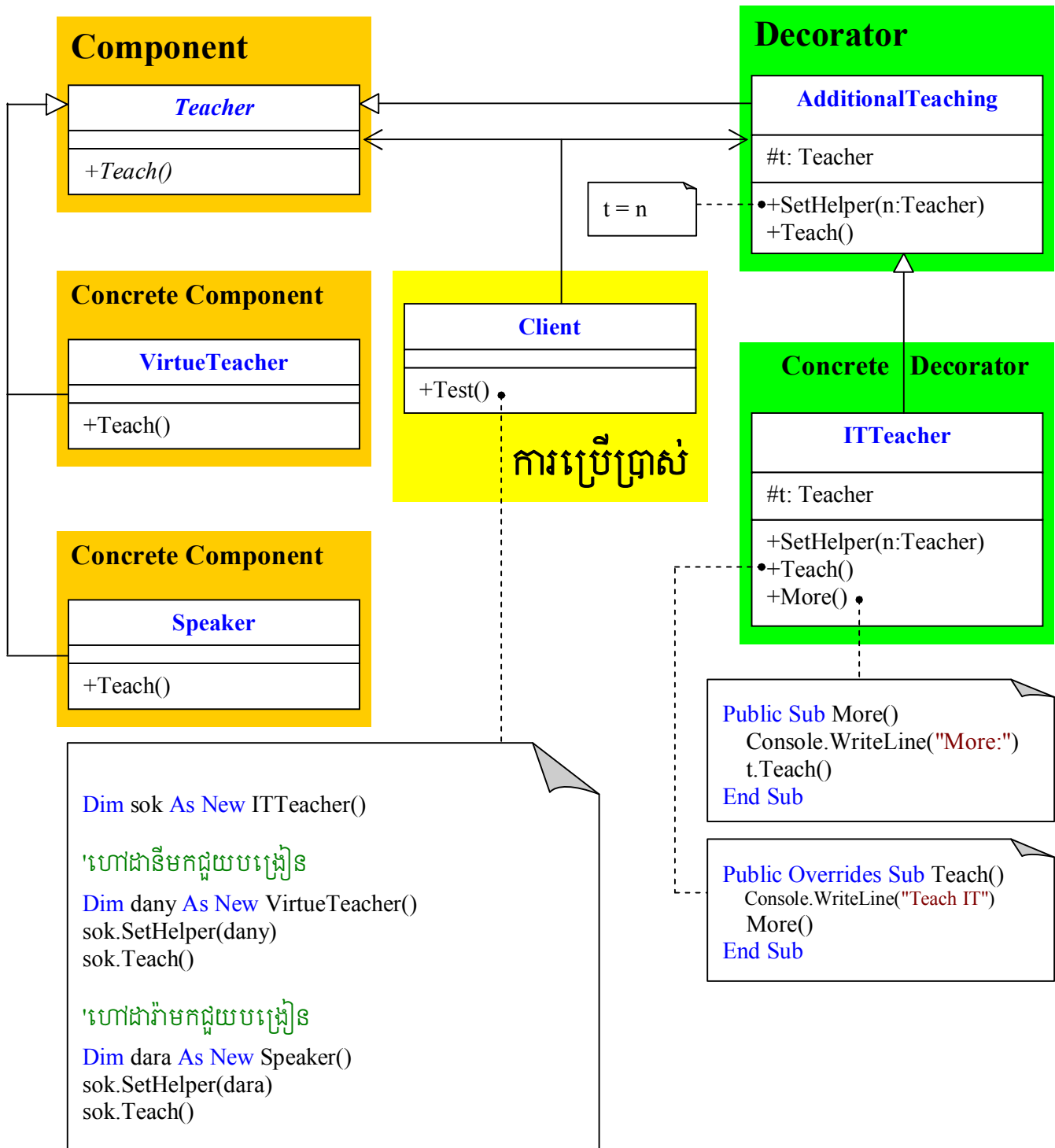


គ្រូព័ត៌មានវិទ្យាអាចបង្រៀនដោយសហការជាមួយគ្រូសុជីវធម៌ដើម្បីជួយ ហើយក៏អាចសហការជាមួយគ្រូផ្សេងៗទៀត។

ដូច្នេះ

ការបន្ថែម ឬ លុប Responsibility នៃ Object ក៏អាចធ្វើបានតាមមធ្យោបាយបង្កើត និងផ្តាច់អន្តរទំនាក់ទំនងជាមួយ Object ប្រភេទដូចគ្នានេះ។

ដក្រង UML នៃគំរូបបន្ថែមលម្អ



Module Decorator

'Abstract Component

Public Class Teacher

Overridable Sub Teach()

End Sub

End Class

'Concrete Component

Public Class VirtueTeacher

Inherits Teacher

Public Overrides Sub Teach()

Console.WriteLine("Teach Virtue.")

End Sub

End Class

'Concrete Component

Public Class Speaker

Inherits Teacher

Public Overrides Sub Teach()

Console.WriteLine("Teach Additional Topic.")

End Sub

End Class

'Abstract Decorator

Public Class AdditionalTeaching

Inherits Teacher

Protected t As Teacher

Public Sub SetHelper(ByRef n As Teacher)

t = n

End Sub

End Class

'Concrete Decorator

Public Class ITTeacher

Inherits AdditionalTeaching

'ការបង្រៀនផ្ទាល់ខ្លួន + ភារកិច្ចបន្ថែម

Public Overrides Sub Teach()

Console.WriteLine("Teach IT")

More()

End Sub

'ភារកិច្ចត្រូវបានបន្ថែមនៅទីនេះតាមរយៈគ្រូដែលបានកំណត់

Public Sub More()

Console.WriteLine("More:")

t.Teach()

End Sub

End Class

```

Public Class Client
    Public Shared Sub Test()
        Dim sok As New ITTeacher()

        'ហៅជានីមកជួយបង្រៀន
        Dim dany As New VirtueTeacher()
        sok.SetHelper(dany)
        sok.Teach()

        Console.WriteLine()

        'ហៅជារ៉ាមកជួយបង្រៀន
        Dim dara As New Speaker()
        sok.SetHelper(dara)
        sok.Teach()

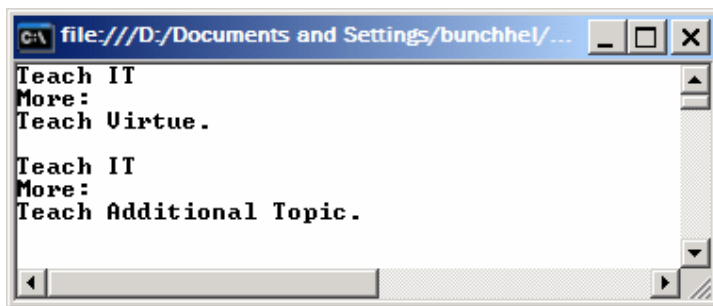
        Console.Read()
    End Sub
End Class

Sub Main()
    Client.Test()
End Sub

```

End Module

លទ្ធផល



គំរូបែបសមាសចម្រុះ

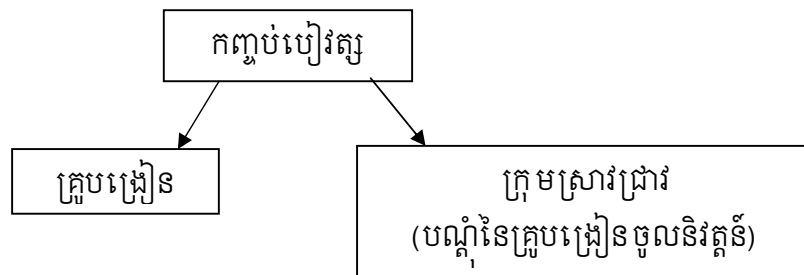
(Composite Pattern)

គោលដៅ

អនុញ្ញាតឱ្យ Client ប្រើប្រាស់ Object ឬក្រុមនៃ Object ដូចជា Object ប្រភេទតែមួយ។

ជីវិតពិត

ផ្តល់បៀវត្សគ្រូបង្រៀន និងក្រុមស្រាវជ្រាវជាកញ្ចប់។

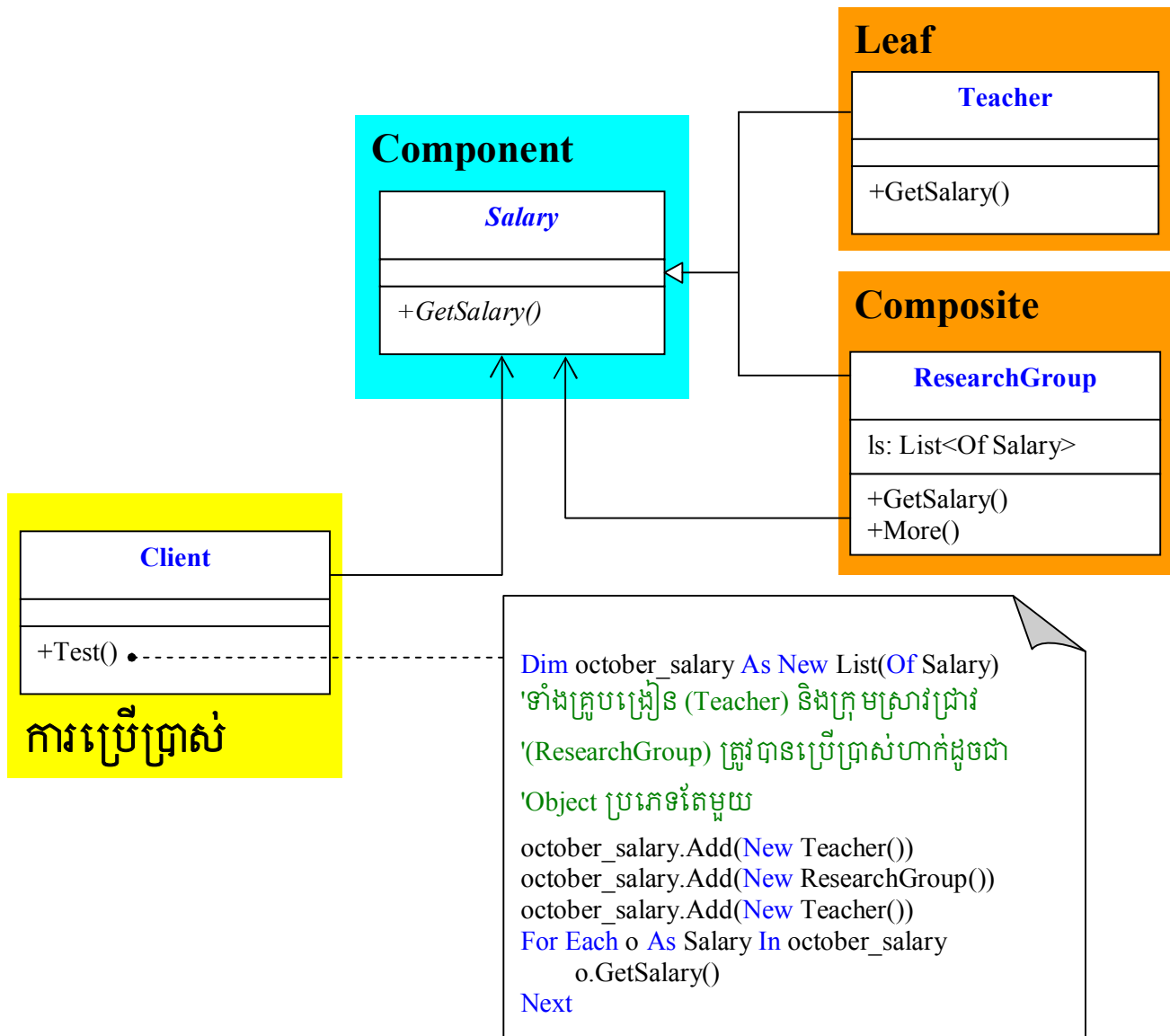


ការផ្តល់បៀវត្សរបៀបនេះធ្វើឱ្យងាយស្រួលដល់អ្នករៀបចំបៀវត្ស

ដូច្នេះ

ការអនុញ្ញាតឱ្យ Client ប្រើប្រាស់ Object ឬក្រុមនៃ Object ដូចជា Object ប្រភេទតែមួយ ក៏ដើម្បីបង្កើនភាពងាយស្រួលដល់ Client ដែរ។

ដ្យាក្រាម UML ដែលបង្ហាញពីសមាសធាតុ:



ការអនុវត្ត

Module Composite

'Component

Public Class Salary

Overridable Sub GetSalary()

End Sub

End Class

'Leaf

Public Class Teacher

Inherits Salary

Public Overrides Sub GetSalary()

Console.WriteLine("Teacher Salary.")

End Sub

End Class

'Composite

Public Class ResearchGroup

Inherits Salary

Private ls As New List(Of Salary)

Public Overrides Sub GetSalary()

'គណនាប្រៀបធៀបដោយបូកសរុបចេញពីសមាជិកទាំងអស់នៅក្នុងបញ្ជី ls

Console.WriteLine("Research Group Salary.")

End Sub

Public Sub More()

End Sub

End Class

Public Class Client

Public Shared Sub Test()

Dim october_salary As New List(Of Salary)

'ទាំងគ្រូបង្រៀន (Teacher) និងក្រុម មស្រាវជ្រាវ (ResearchGroup)

'ត្រូវបានប្រើប្រាស់ហាក់ដូចជា Object ប្រភេទតែមួយ

october_salary.Add(New Teacher())

october_salary.Add(New ResearchGroup())

october_salary.Add(New Teacher())

For Each o As Salary In october_salary

o.GetSalary()

Next

Console.Read()

End Sub

End Class

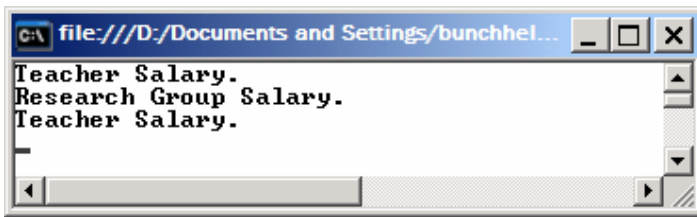
Sub Main()

Client.Test()

End Sub

End Module

លទ្ធផល



ប្រភេទគំរូមែឃប្រព្រឹត្ត (Behavioral Pattern)

គំរូបែបវដ្តដំណើរ (Iterator Pattern)

គោលដៅ

Access ទៅលើសំណុំនៃ Object ដោយប្រយោល។

ជីវិតពិត

គ្រូប្រើប្រាស់បញ្ជីនិស្សិតដើម្បីគ្រប់គ្រងក្នុងថ្នាក់។

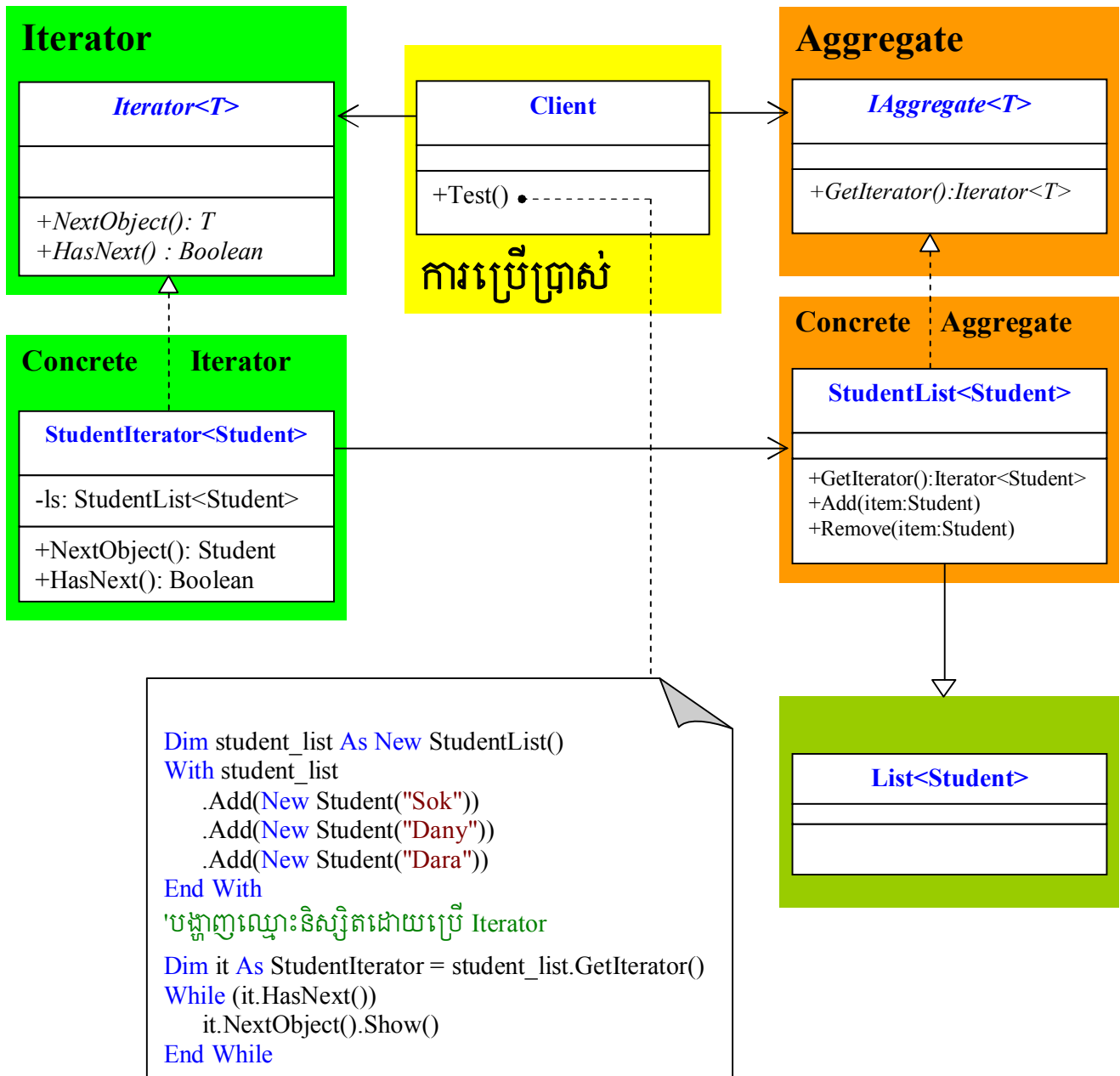
បញ្ជីវត្ថុមាន
១ - សុខ
២ - ដានី
៣ - ដារ៉ា

បញ្ជីជាមធ្យោបាយដែលគ្រូ Access ទៅកាន់និស្សិតដោយប្រយោល។

ដូច្នេះ

Iterator ផ្តល់មធ្យោបាយក្នុងការ Access ដោយប្រយោលទៅលើ Object នៅក្នុងសំណុំ។

ដ្យាក្រាម UML នៃគំរូបេបវដ្តដំណើរ



Module Iterator

'Object ដែលត្រូវ Access

```
Public Class Student
    Private name As String

    Public Sub New(ByVal value As String)
        name = value
    End Sub

    Public Sub Show()
        Console.WriteLine(name)
    End Sub
End Class
```

'Aggregate

```
Public Interface IAggregate(Of T)
    Function GetIterator() As T
End Interface
```

'Concrete Aggregate

```
Public Class StudentList
    Inherits List(Of Student)
    Implements IAggregate(Of StudentIterator)

    Public Function GetIterator() As StudentIterator Implements IAggregate(Of StudentIterator).GetIterator
        Return New StudentIterator(Me)
    End Function
End Class
```

'Iterator

```
Public Interface Iterator(Of T)
    Function NextObject() As T
    Function HasNext() As Boolean
End Interface
```

'Concrete Iterator

```
Public Class StudentIterator
    Implements Iterator(Of Student)

    Private ls As StudentList
    Private current As Integer

    Public Sub New(ByVal value As StudentList)
        ls = value
    End Sub

    Public Function HasNext() As Boolean Implements Iterator(Of Student).HasNext
        If current < ls.Count Then
            Return True
        Else
            Return False
        End If
    End Function
End Class
```

End Function

```
Public Function NextObject() As Student Implements Iterator(Of Student).NextObject
```

```
    Dim st As Student = ls(current)
```

```
    current += 1
```

```
    Return st
```

```
End Function
```

```
End Class
```

```
Public Class Client
```

```
    Public Shared Sub Test()
```

```
        Dim student_list As New StudentList()
```

```
        With student_list
```

```
            .Add(New Student("Sok"))
```

```
            .Add(New Student("Dany"))
```

```
            .Add(New Student("Dara"))
```

```
        End With
```

```
        'បង្ហាញឈ្មោះនិស្សិតដោយប្រើ Iterator
```

```
        Dim it As StudentIterator = student_list.GetIterator()
```

```
        While (it.HasNext())
```

```
            it.NextObject().Show()
```

```
        End While
```

```
        Console.Read()
```

```
    End Sub
```

```
End Class
```

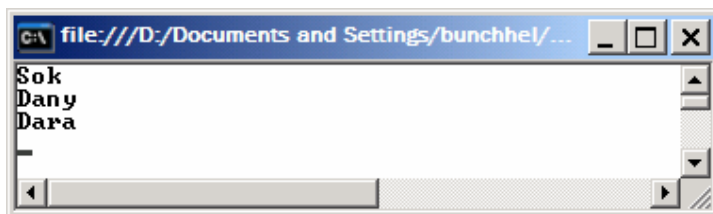
```
Sub Main()
```

```
    Client.Test()
```

```
End Sub
```

```
End Module
```

លទ្ធផល



តំរូវបែបស្វ័យគ្រប់គ្រង

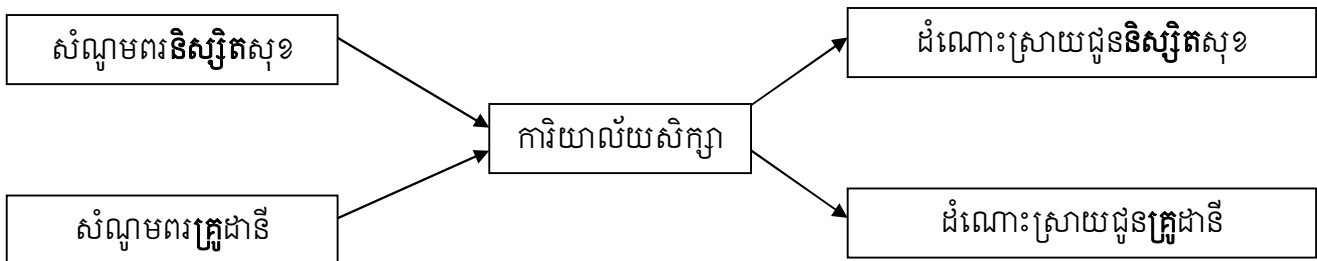
(Command Pattern)

គោលដៅ

វិច្ចប់សំណូមពរ (Request) ក្នុង Object និងរក្សាទុកសម្រាប់ដំណើរការនៅពេលក្រោយ។

ជីវិតពិត

សំណូមពរនៃនិស្សិត និងគ្រូ ជាមួយឯកសារយោងត្រូវបានដាក់ក្នុងស្រោមសំបុត្រនីមួយៗ ដើម្បីឱ្យការិយាល័យសិក្សាដោះស្រាយតាមកាលៈទេសៈ។

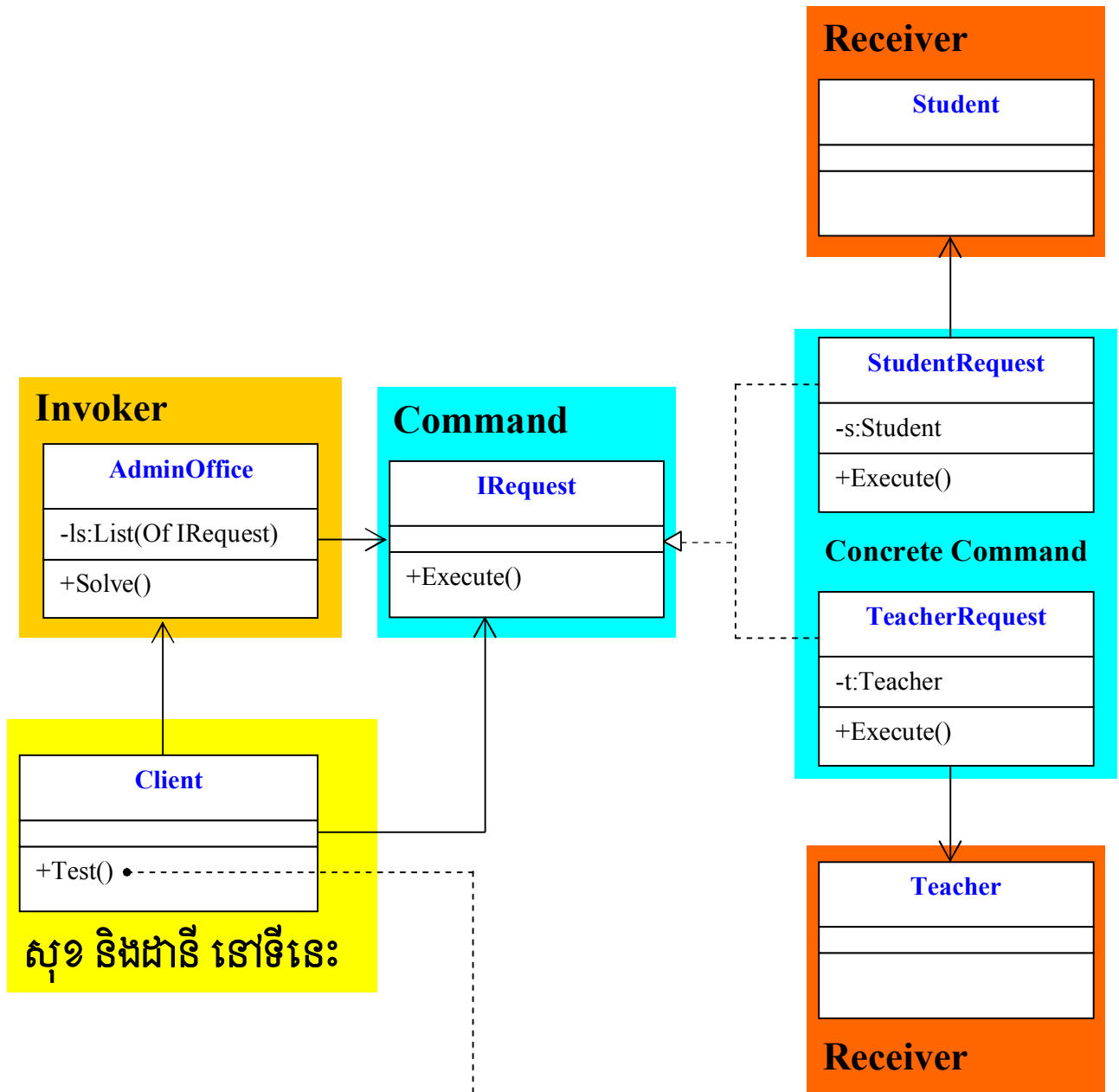


ការិយាល័យសិក្សាបើកស្រោមសំបុត្រនីមួយៗដើម្បីដោះស្រាយសំណូមពរ ទៅតាមកាលៈទេសៈសមស្របដោយសំអាងលើឯកសារយោងផង។ សំណូមពរទាំងនេះត្រូវបានដោះស្រាយដោយពុំចាំបាច់មាននិស្សិតនិងគ្រូនៅផ្ទាល់ជាមួយឡើយ។

ដូច្នេះ

ការវិច្ចប់សំណូមពរ (Request) ក្នុង Object ដើម្បីអាចផ្ទុកនូវ Property ផ្សេងៗពាក់ព័ន្ធនឹងសំណូមពរផង និងអាចរក្សាទុកសម្រាប់ដំណើរការនៅពេលក្រោយ ដោយ Client ដែលសំណូមពរអាចធ្វើការងារផ្សេង ដោយទុកឱ្យសំណូមពរនោះអាចដោះស្រាយទៅតាមកាលៈទេសៈសមស្រប។

ដ្យាក្រាម UML នៃគំរូបប្រែប្រួលស្វ័យគ្រប់គ្រង



```

Dim teacher_request_list As New List(Of IRequest)
'សំណូមពររបស់ដានីត្រូវបានរក្សាទុកនៅក្នុងបញ្ជីសំណូមពរគ្រូ
Dim dany As New Teacher()
Dim tr As New TeacherRequest(dany)
teacher_request_list.Add(tr)
'ដោះស្រាយសំណូមពរទាំងឡាយរបស់គ្រូ
Dim admin As New AdminOffice()
admin.SetInvoker(teacher_request_list)
admin.Solve()
    
```

Module Command

'Receiver

Public Class Student

End Class

'Receiver

Public Class Teacher

End Class

'Command

Public Interface IRequest

Sub Execute()

End Interface

'Concrete Command

Public Class StudentRequest

Implements IRequest

Private s As Student

Public Sub New(ByRef value As Student)

s = value

End Sub

Public Sub Execute() Implements IRequest.Execute

Console.WriteLine("Show this student's transcript by getting from database.")

End Sub

End Class

'Concrete Command

Public Class TeacherRequest

Implements IRequest

Private t As Teacher

Public Sub New(ByRef value As Teacher)

t = value

End Sub

Public Sub Execute() Implements IRequest.Execute

Console.WriteLine("Update this teacher info to database.")

End Sub

End Class

'Invoker

Public Class AdminOffice

Private ls As List(Of IRequest)

Public Sub SetInvoker(ByRef value As List(Of IRequest))

ls = value

End Sub

Public Sub Solve()

For Each o As IRequest In ls

```

        o.Execute()
    Next
End Sub
End Class

Public Class Client
    Public Shared Sub Test()
        Dim student_request_list As New List(Of IRequest)
        Dim teacher_request_list As New List(Of IRequest)

        'សំណូមពររបស់សុខត្រូវបានរក្សាទុកនៅក្នុងបញ្ជីសំណូមពរនិស្សិត
        Dim sok As New Student()
        Dim sr As New StudentRequest(sok)
        student_request_list.Add(sr)

        'សំណូមពររបស់ដានីត្រូវបានរក្សាទុកនៅក្នុងបញ្ជីសំណូមពរគ្រូ
        Dim dany As New Teacher()
        Dim tr As New TeacherRequest(dany)
        teacher_request_list.Add(tr)

        'សំណូមពររបស់ដារ៉ាត្រូវបានរក្សាទុកនៅក្នុងបញ្ជីសំណូមពរនិស្សិត
        Dim dara As New Student()
        Dim sr1 As New StudentRequest(dara)
        student_request_list.Add(sr1)

        'ដោះស្រាយសំណូមទាំងឡាយរបស់និស្សិត
        Dim admin As New AdminOffice()
        admin.SetInvoker(student_request_list)
        admin.Solve()

        'ដោះស្រាយសំណូមទាំងឡាយរបស់គ្រូ
        admin.SetInvoker(teacher_request_list)
        admin.Solve()

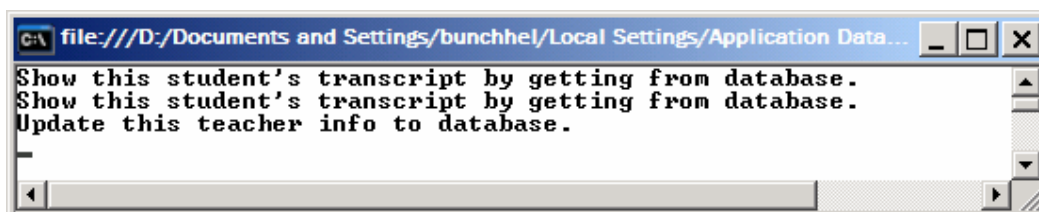
        Console.Read()
    End Sub
End Class

Sub Main()
    Client.Test()
End Sub

End Module

```

លទ្ធផល



គំរូបែបអេក្យូចំណាំ

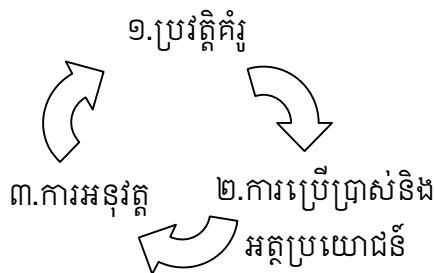
(Memento Pattern)

គោលដៅ

បង្កើត Object ជា Read-Only ដើម្បីផ្ទុកទិន្នន័យនៃ Object ដើមប្រភេទណាមួយ។

ជីវិតពិត

ប្រើការចងចាំដើម្បីត្រឡប់មកដំណាក់កាលណាមួយនៃការបង្រៀន។

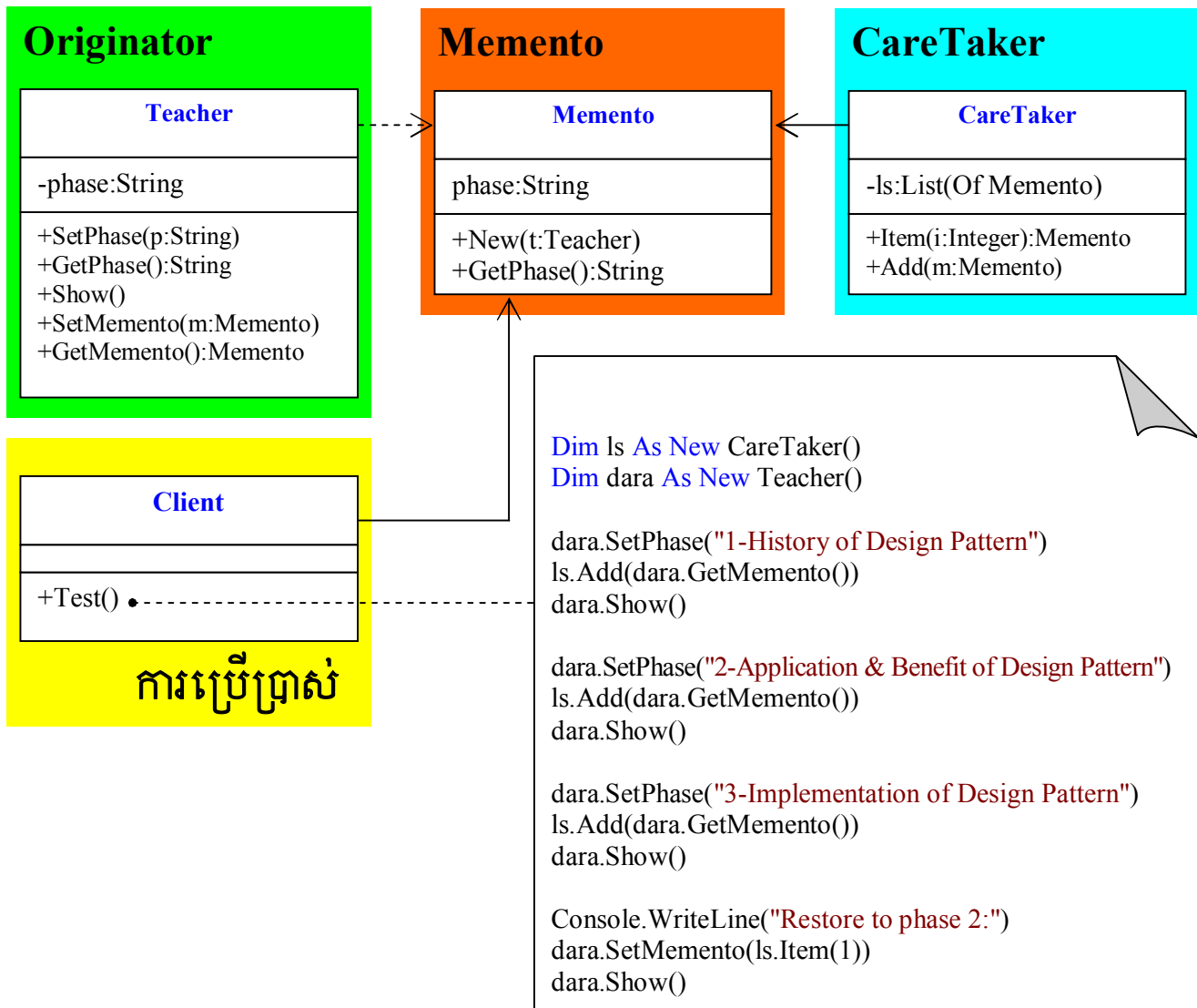


ឧទាហរណ៍ថាគ្រូម្នាក់បង្រៀនកំរុំនៃការរៀបចំកូដណាមួយ គាត់បង្រៀនតាមលំនាំនិយាយពីប្រវត្តិនៃកំរុំ បន្ទាប់មកការប្រើប្រាស់និងអត្ថប្រយោជន៍ និងចុងក្រោយការអនុវត្តនៃកំរុំ។ នៅពេលបញ្ចប់គាត់ បង្រៀនសាដើមវិញ ឬត្រឡប់ទៅដំណាក់កាលណាមួយដើម្បីបញ្ជាក់បន្ថែម។

ដូច្នេះ

បង្កើត Object ជា Read-Only ដើម្បីផ្ទុកទិន្នន័យនៃ Object ដើមប្រភេទណាមួយគឺដើម្បីអាចឱ្យ Object ដើមនោះ Restore ទិន្នន័យរបស់ខ្លួនមកដំណាក់កាលណាមួយ ដោយប្រើប្រាស់ Read-Only Object នោះ។ គេប្រើវាជាពិសេសចំពោះសកម្មភាព Undo និង Redo។

ដ្យាក្រាម UML នៃគំរូបបញ្ចប់ការងារ



ការអនុវត្ត

Module Memento_Demo

'Originator

Public Class Teacher

Private phase As String

Public Sub SetPhase(ByVal value As String)

phase = value

End Sub

Public Function GetPhase() As String

Return phase

End Function

Public Sub Show()

Console.WriteLine(phase)

End Sub

'Return នូវដំណាក់កាលកំពុងបង្រៀនតាមរយៈ Memento

Public Function GetMemento() As Memento

Return New Memento(Me)

End Function

'Restore ដំណាក់កាលបង្រៀនទៅតាម Memento ដែលផ្តល់ឱ្យ

Public Sub SetMemento(ByRef m As Memento)

phase = m.GetPhase()

End Sub

End Class 'End Class Teacher

'Memento

Public Class Memento

Private phase As String

'ទទួលទិន្នន័យពី Teacher ដើម្បីរក្សាទុក

Public Sub New(ByRef t As Teacher)

phase = t.GetPhase()

End Sub

Public Function GetPhase() As String

Return phase

End Function

End Class 'End Class Memento

'CareTaker

'ជា List សម្រាប់រក្សាទុក Memento ទាំងឡាយ

Public Class CareTaker

Inherits List(Of Memento)

End Class

Public Class Client

Public Shared Sub Test()

Dim ls As New CareTaker()

Dim dara As New Teacher()

'ដំណាក់កាលទី១នៃការបង្រៀននោះគឺបង្រៀនពីប្រវត្តិកំរូនៃការរៀបចំកូដ

'កត់ដំណាក់កាលនេះទុកនៅក្នុង ls ដែលជា CareTaker

dara.SetPhase("1-History of Design Pattern")

ls.Add(dara.GetMemento())

dara.Show()

'ដំណាក់កាលទី២នៃការបង្រៀននោះគឺបង្រៀនពីការប្រើប្រាស់ និងផលប្រយោជន៍កំរូនៃការរៀបចំកូដ

'កត់ដំណាក់កាលនេះទុកនៅក្នុង ls ដែលជា CareTaker

dara.SetPhase("2-Application & Benefit of Design Pattern")

ls.Add(dara.GetMemento())

dara.Show()

'ដំណាក់កាលទី៣នៃការបង្រៀននោះគឺបង្រៀនពីការអនុវត្តកំរូនៃការរៀបចំកូដ

'កត់ដំណាក់កាលនេះទុកនៅក្នុង ls ដែលជា CareTaker

dara.SetPhase("3-Implementation of Design Pattern")

ls.Add(dara.GetMemento())

dara.Show()

'Restore ទៅដំណាក់កាលទី២នៃការបង្រៀននោះគឺនិយាយពីការប្រើប្រាស់ និងផលប្រយោជន៍

'នៃកំរូនៃការរៀបចំកូដ

Console.WriteLine()

Console.WriteLine("Restore to phase 2:")

dara.SetMemento(ls.Item(1))

dara.Show()

'Restore ទៅដំណាក់កាលទី៣នៃការបង្រៀននោះគឺការអនុវត្តនៃកំរូនៃការរៀបចំកូដ

Console.WriteLine()

Console.WriteLine("Restore to phase 3:")

dara.SetMemento(ls.Item(2))

dara.Show()

Console.Read()

End Sub

End Class

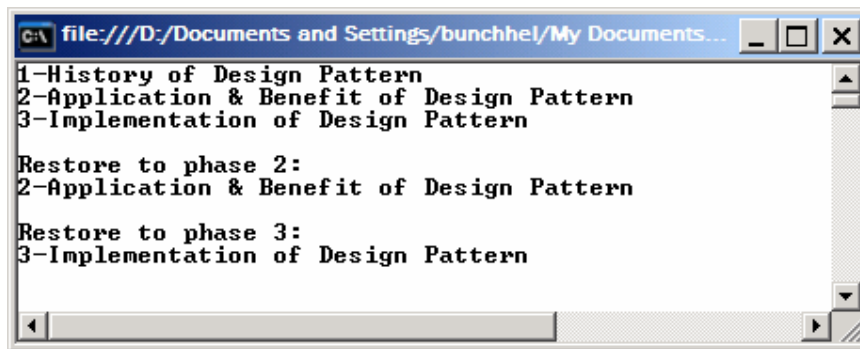
Sub Main()

Client.Test()

End Sub

End Module

លទ្ធផល



គំរូបែបសណ្ឋានលក្ខខណ្ឌ

(State Pattern)

គោលដៅ

អនុញ្ញាតឱ្យ Object ប្តូរនូវអាកប្បកិរិយា និងមុខងាររបស់ខ្លួនអាស្រ័យទៅលើតម្លៃផ្ទៃក្នុងរបស់វា។

ជីវិតពិត

គ្រូបង្រៀននិស្សិតអាស្រ័យលើសញ្ញាបត្ររបស់ខ្លួន។

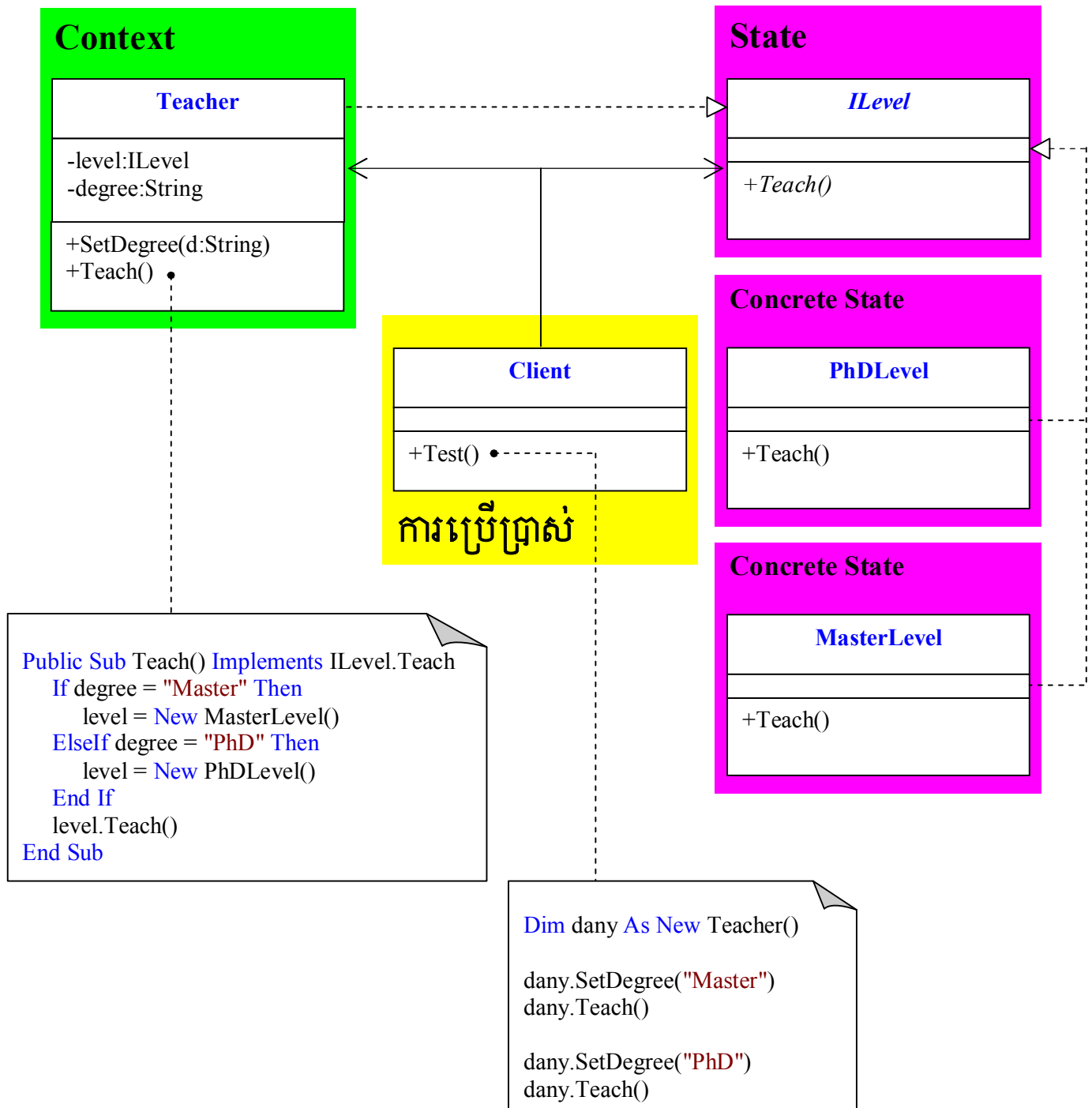


ការអនុវត្តតាមរបៀបនេះធ្វើឱ្យគ្រូអាចផ្លាស់ប្តូរកម្រិតបង្រៀន ទៅតាមសញ្ញាបត្រដែលខ្លួនមាន។

ដូច្នេះ

Object អាចប្តូរនូវអាកប្បកិរិយា និងមុខងាររបស់ខ្លួនអាស្រ័យទៅលើតម្លៃផ្ទៃក្នុងរបស់វា។

ដ្យាក្រាម UML នៃគំរូមេបសណ្ឋានលក្ខខណ្ឌ



Module State

'State

Public Interface ILevel

Sub Teach()

End Interface

'Concrete State

Public Class PhDLevel

Implements ILevel

Public Sub Teach() Implements ILevel.Teach

Console.WriteLine("PhD Level: Teaches Master Level")

End Sub

End Class

'Concrete State

Public Class MasterLevel

Implements ILevel

Public Sub Teach() Implements ILevel.Teach

Console.WriteLine("Master Level: Teaches Bachelor Level")

End Sub

End Class

'Context

Public Class Teacher

Implements ILevel

Private level As ILevel

Private degree As String

Public Sub SetDegree(ByVal value As String)

degree = value

End Sub

Public Sub Teach() Implements ILevel.Teach

If degree = "Master" Then

level = New MasterLevel()

ElseIf degree = "PhD" Then

level = New PhDLevel()

End If

level.Teach()

End Sub

End Class

Public Class Client

Public Shared Sub Test()

Dim dany As New Teacher()

dany.SetDegree("Master")

dany.Teach()

```
dany.SetDegree("PhD")  
dany.Teach()
```

```
Console.Read()
```

```
End Sub
```

```
End Class
```

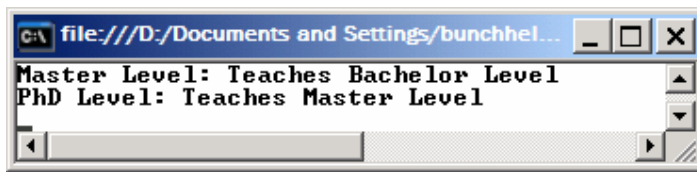
```
Sub Main()
```

```
Client.Test()
```

```
End Sub
```

```
End Module
```

លទ្ធផល



តំរូវបែបខ្នាតពុម្ព

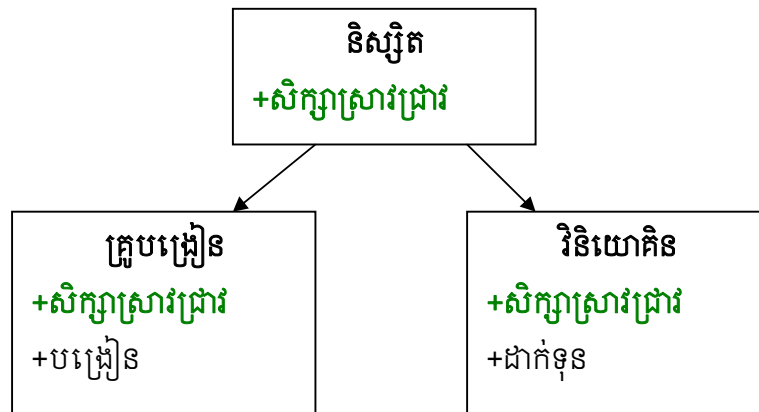
(Template Method Pattern)

គោលដៅ

កំណត់នូវគ្រោងឆ្លឹង និងពុម្ពនៃ Class ដើម សម្រាប់ការបន្តមរតកប្រើប្រាស់ទៅ Class ជំនាន់ក្រោយ។

ជីវិតពិត

ជីវិតគ្រូបង្រៀន ឬវិនិយោគិនទទួលមរតកពីជីវិតជានិស្សិត។

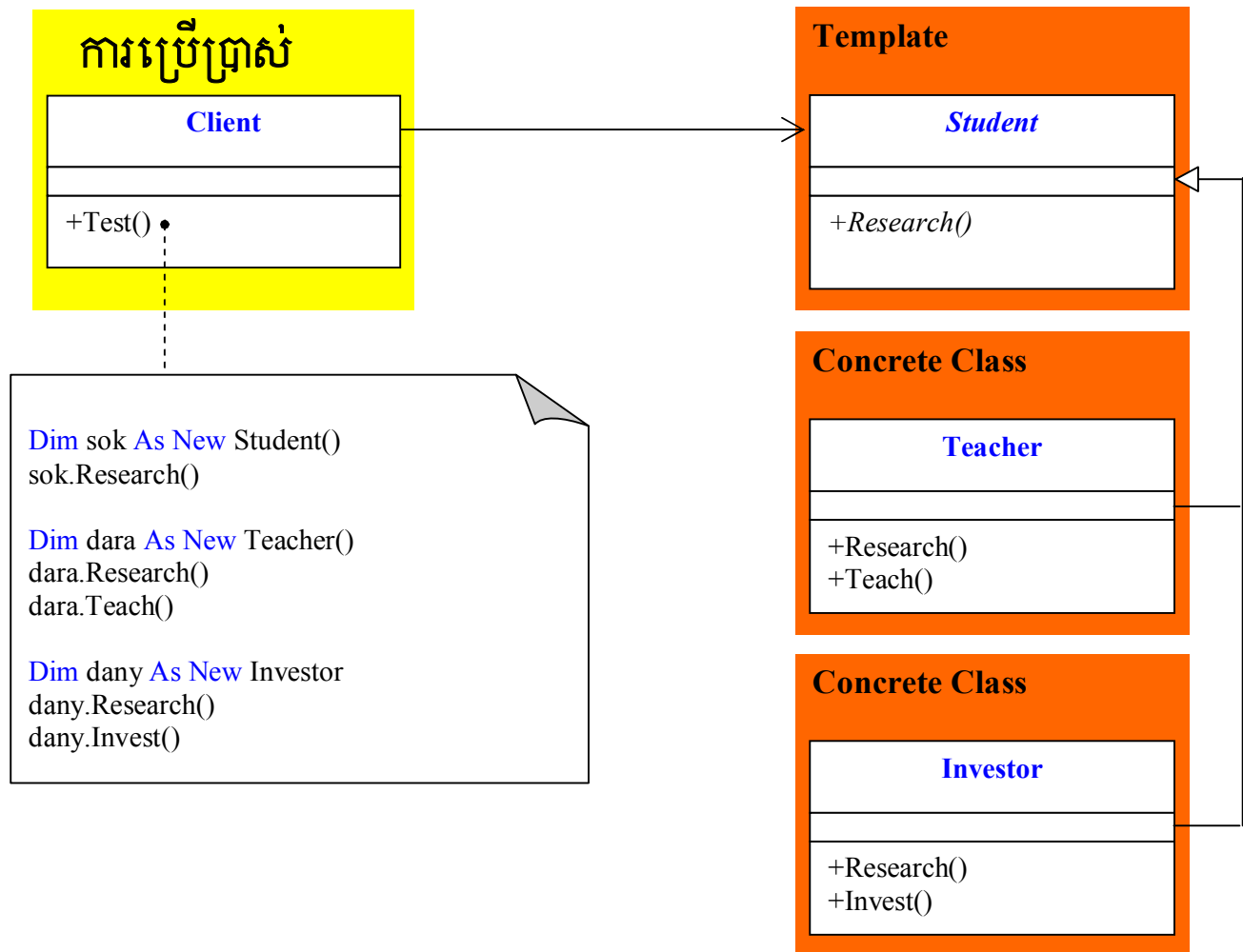


ការសិក្សាស្រាវជ្រាវជាមរតកតាំងពីនៅជានិស្សិតរហូតដល់ក្លាយជាគ្រូបង្រៀន និងវិនិយោគិនដោយគ្រាន់តែគ្រូបង្រៀន និងវិនិយោគិនបន្ថែមនូវមុខជំនាញផ្ទាល់ខ្លួនផ្សេងទៀតប៉ុណ្ណោះ។

ដូច្នេះ

គ្រោងឆ្លឹង និងពុម្ពនៃ Class ដើមអាចត្រូវបានប្រើប្រាស់បន្ត និងអាចបន្ថែមនូវលក្ខណៈពិសេសផ្ទាល់មួយចំនួនទៀតទៅតាមប្រភេទនៃ Class ជំនាន់ក្រោយ។

ជំពូក UML នៃគំរូបបង្កាត់កុម្មុ



ការអនុវត្ត

Module Template

'Template

Public Class Student

Public Overridable Sub Research()

Console.WriteLine("Research Ability")

End Sub

End Class

'Concrete Class

Public Class Teacher

Inherits Student

Public Sub Teach()

Console.WriteLine("Teaching Ability")

End Sub

End Class

'Concrete Class

Public Class Investor

Inherits Student

Public Sub Invest()

Console.WriteLine("Investing Ability")

End Sub

End Class

Public Class Client

Public Shared Sub Test()

'និស្សិតមានការសិក្សាស្រាវជ្រាវ

Console.WriteLine("As a student: ")

Dim sok As New Student()

sok.Research()

'គ្រូបង្រៀនក៏មានការសិក្សាស្រាវជ្រាវដែលជាមរតកកាលពីនៅជានិស្សិត

'ព្រមទាំងមានសមត្ថភាពនៃការបង្រៀន

Console.WriteLine()

Console.WriteLine("As a teacher: ")

Dim dara As New Teacher()

dara.Research()

dara.Teach()

'វិនិយោគិនក៏មានការសិក្សាស្រាវជ្រាវដែលជាមរតកកាលពីនៅជានិស្សិត

'ព្រមទាំងមានសមត្ថភាពនៃការបណ្តាក់ទុន

Console.WriteLine()

Console.WriteLine("As an investor: ")

Dim dany As New Investor

dany.Research()

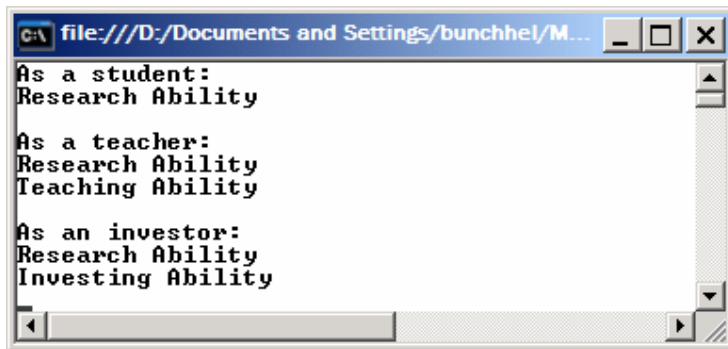
dany.Invest()

```
        Console.Read()  
    End Sub  
End Class
```

```
Sub Main()  
    Client.Test()  
End Sub
```

```
End Module
```

លទ្ធផល



គំរូបែបយុទ្ធសាស្ត្រ

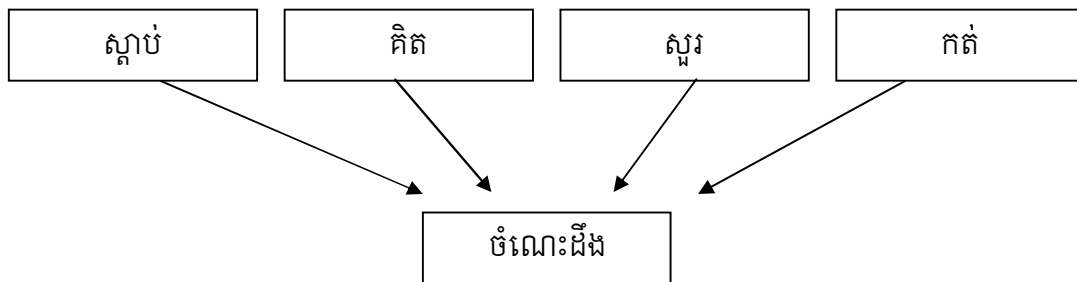
(Strategy Pattern)

គោលដៅ

វិធានប្រើ Algorithm នៅក្នុងយុទ្ធសាស្ត្រផ្សេងៗគ្នា ដើម្បីប្រើប្រាស់សម្រេចគោលបំណងជាក់លាក់។

ជីវិតពិត

យុទ្ធសាស្ត្រដើម្បីទទួលបានចំណេះដឹង។

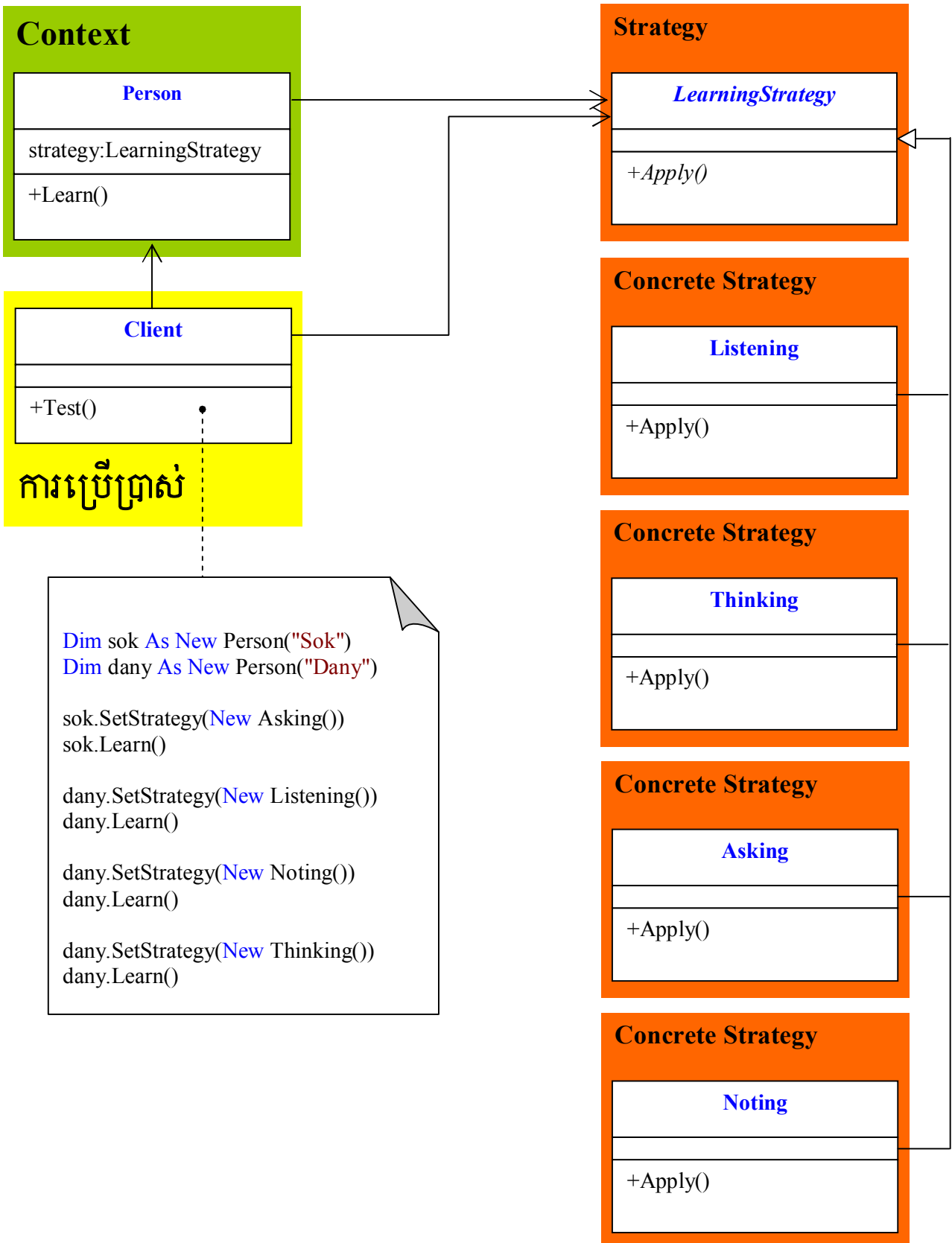


យុទ្ធសាស្ត្រនេះអាចត្រូវបានប្រើប្រាស់ដោយជនទាំងឡាយ ។ល។

ដូច្នេះ

គំរូនេះស្រដៀងទៅនឹងគំរូបែបសណ្ឋានលក្ខខណ្ឌ (State Pattern) ប៉ុន្តែគំរូបែបយុទ្ធសាស្ត្រនេះ ជាទូទៅប្រើប្រាស់ទូលាយជាងដោយអនុវត្តបានចំពោះ Object ច្រើនប្រភេទ។

ជំពូក UML ដែលបង្ហាញពីការប្រើប្រាស់



Module Strategy

```

'Strategy
Public Class LearningStrategy
    Public Overridable Sub Apply()
    End Sub
End Class

'Concrete Strategy
Public Class Listening
    Inherits LearningStrategy
    Public Overrides Sub Apply()
        Console.WriteLine("Listening")
    End Sub
End Class

'Concrete Strategy
Public Class Thinking
    Inherits LearningStrategy
    Public Overrides Sub Apply()
        Console.WriteLine("Thinking")
    End Sub
End Class

'Concrete Strategy
Public Class Asking
    Inherits LearningStrategy
    Public Overrides Sub Apply()
        Console.WriteLine("Asking")
    End Sub
End Class

'Concrete Strategy
Public Class Noting
    Inherits LearningStrategy
    Public Overrides Sub Apply()
        Console.WriteLine("Noting")
    End Sub
End Class

'Context
Public Class Person
    Private strategy As LearningStrategy
    Private name As String

    Public Sub New(ByVal value As String)
        name = value
    End Sub

    Public Sub SetStrategy(ByRef value As LearningStrategy)
        strategy = value
    End Sub

```

```

Public Sub Learn()
    Console.Write(name)
    Console.Write(" ")
    strategy.Apply()
    Console.WriteLine()
End Sub
End Class

Public Class Client
    Public Shared Sub Test()
        Dim sok As New Person("Sok")
        Dim dany As New Person("Dany")

        sok.SetStrategy(New Asking())
        sok.Learn()

        dany.SetStrategy(New Listening())
        dany.Learn()

        dany.SetStrategy(New Noting())
        dany.Learn()

        dany.SetStrategy(New Thinking())
        dany.Learn()

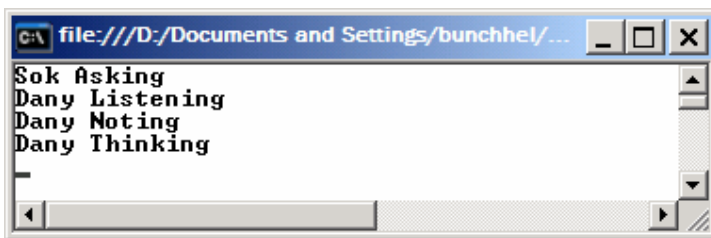
        Console.Read()
    End Sub
End Class

Sub Main()
    Client.Test()
End Sub

End Module

```

លទ្ធផល



គំរូបែបសង្កេតការណ៍

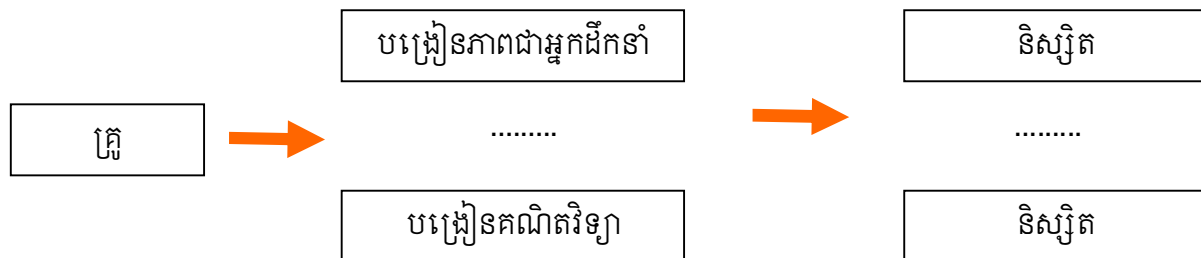
(Observer Pattern)

គោលដៅ

អនុញ្ញាតឱ្យកំណែប្រែនៃសណ្ឋាន ឬ លក្ខណៈ របស់ Class ដើម អាចបញ្ជូនទៅកាន់ Class ជំនាន់ក្រោយបាន។

ជីវិតពិត

គ្រូបង្រៀនប្រធានបទណាមួយដល់និស្សិតទាំងឡាយ។

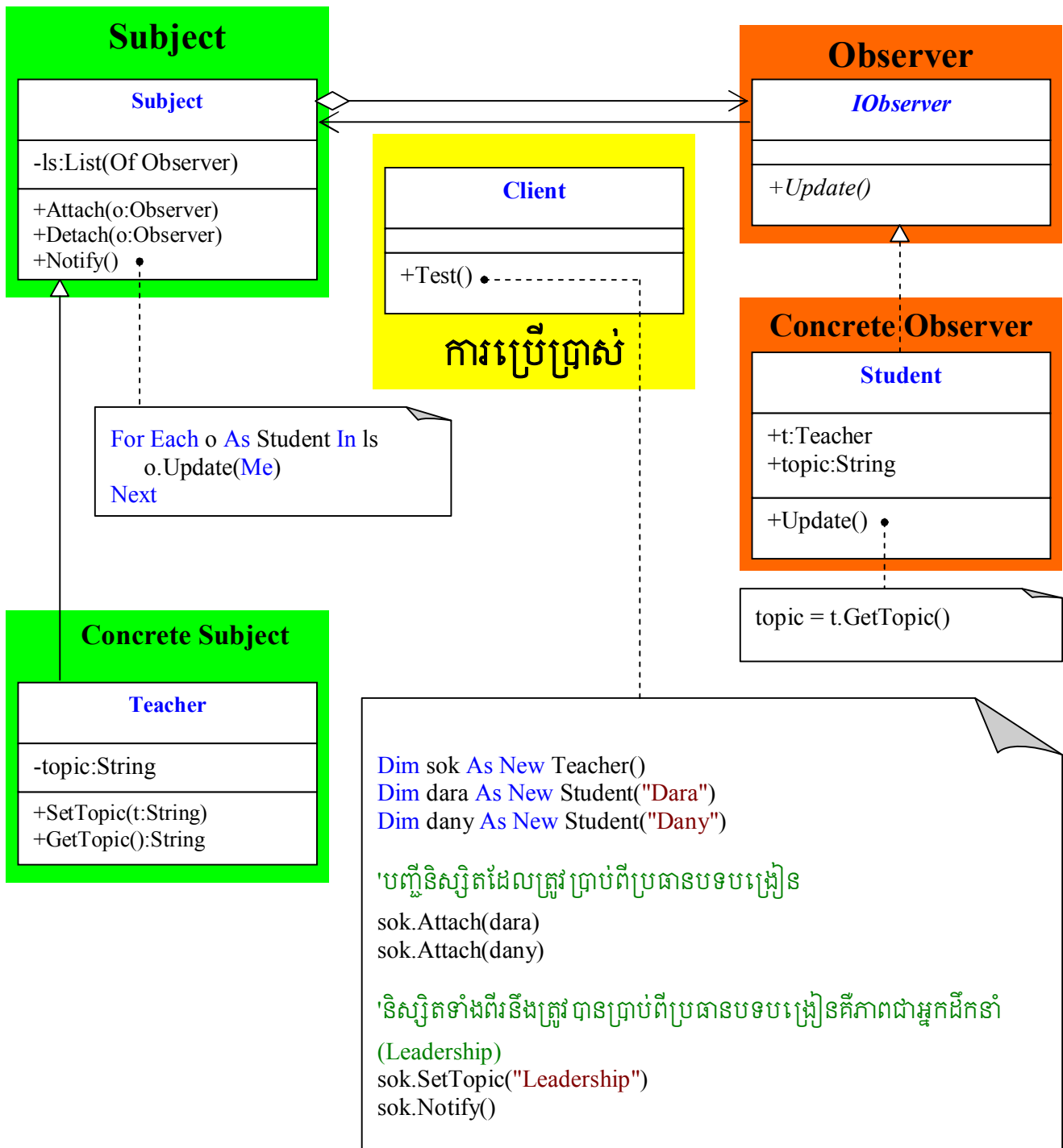


និស្សិតទាំងឡាយដឹងពីប្រធានបទដែលគ្រូបានបង្រៀន។

ដូច្នេះ

Class ជំនាន់ក្រោយ ក៏អាចដឹងពីព័ត៌មានដែល Class ដើមបានផ្តល់ឱ្យ។

ជួរក្រាម UML នៃគំរូបសង្កេតការណ៍



Module Observer

'Observer

Public Interface IObserver

Sub Update(ByRef t As Teacher)

End Interface

'Concrete Observer

Public Class Student

Implements IObserver

Private topic As String = "unknown"

Private name As String

Public Sub New(ByVal value As String)

name = value

End Sub

Public Sub Update(ByRef t As Teacher) Implements IObserver.Update

topic = t.GetTopic()

End Sub

Public Sub ShowTopic()

Console.WriteLine(name + " said the topic was: " + topic)

End Sub

End Class

'Subject

Public Class Subject

Private ls As New List(Of Student)

Public Sub Attach(ByRef o As Student)

ls.Add(o)

End Sub

Public Sub Detach(ByRef o As Student)

ls.Remove(o)

End Sub

Public Sub Notify()

For Each o As Student In ls

o.Update(Me)

Next

End Sub

End Class

'Concrete Subject

Public Class Teacher

Inherits Subject

Private topic As String

Public Sub SetTopic(ByVal t As String)

topic = t

End Sub

```

Public Function GetTopic() As String
    Return topic
End Function
End Class

```

```

Public Class Client
    Public Shared Sub Test()
        Dim sok As New Teacher()
        Dim dara As New Student("Dara")
        Dim dany As New Student("Dany")

```

'មុនពេលការប្រាប់ពីប្រធានបទ

```

Console.WriteLine("Before Notification:")
dara.ShowTopic()
dany.ShowTopic()

```

'បញ្ជីនិស្សិតដែលត្រូវប្រាប់ពីប្រធានបទបង្រៀន

```

sok.Attach(dara)
sok.Attach(dany)

```

'និស្សិតទាំងពីរនឹងត្រូវបានប្រាប់ពីប្រធានបទបង្រៀនគឺភាពជាអ្នកដឹកនាំ (Leadership)

```

sok.SetTopic("Leadership")
sok.Notify()

```

'បង្ហាញប្រធានបទដែលនិស្សិតទាំងពីរបានទទួលព័ត៌មាន

```

Console.WriteLine()
Console.WriteLine("After Notification about Leadership Topic:")
dara.ShowTopic()
dany.ShowTopic()

```

'លុបដានីចេញពីបញ្ជីនៃនិស្សិតដែលត្រូវប្រាប់ពីប្រធានបទបង្រៀន

```

Console.WriteLine()
Console.WriteLine("Detached Dany From the Notification List")
sok.Detach(dany)

```

'ដានីពុំទទួលព័ត៌មានពីការប្តូរប្រធានបទមកគណិតវិទ្យាទេដោយខ្លួនត្រូវបានដកចេញពីបញ្ជី

```

sok.SetTopic("Mathematic")
sok.Notify()

```

'បង្ហាញប្រធានបទដែលនិស្សិតទាំងពីរបានទទួលព័ត៌មាន

```

Console.WriteLine()
Console.WriteLine("After Notification about Mathematic Topic:")
dara.ShowTopic()
dany.ShowTopic()

```

```

Console.Read()

```

```

End Sub
End Class

```

```

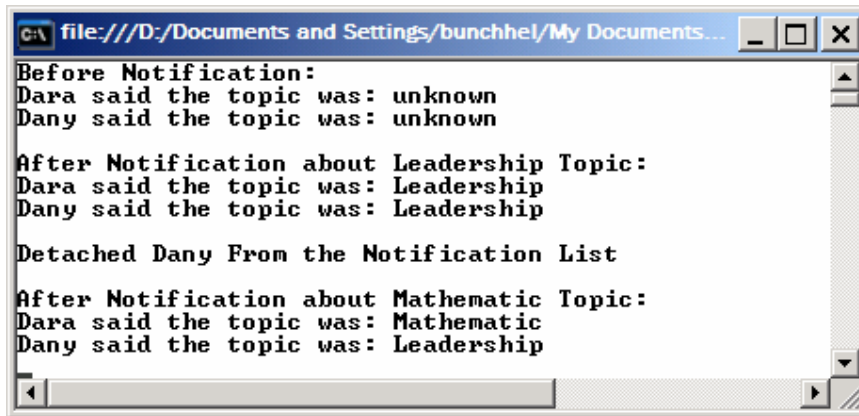
Sub Main()

```

```
Client.Test()  
End Sub
```

```
End Module
```

លទ្ធផល



```
file:///D:/Documents and Settings/bunchhel/My Documents...  
Before Notification:  
Dara said the topic was: unknown  
Dany said the topic was: unknown  
  
After Notification about Leadership Topic:  
Dara said the topic was: Leadership  
Dany said the topic was: Leadership  
  
Detached Dany From the Notification List  
  
After Notification about Mathematic Topic:  
Dara said the topic was: Mathematic  
Dany said the topic was: Leadership
```

គំរូបែបទេសចរណ៍

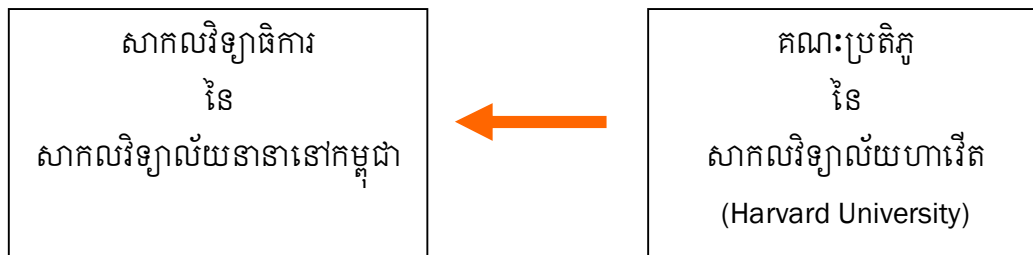
(Visitor Pattern)

គោលដៅ

កំណត់នូវប្រតិបត្តិការថ្មីដោយមិនប៉ះពាល់ដល់ Class ជាតួអង្គផ្ទុកនូវលក្ខណៈទាំងឡាយ។

ជីវិតពិត

ទស្សនកិច្ចនៃតំណាងសាកលវិទ្យាល័យមួយទៅសាកលវិទ្យាល័យជាច្រើនផ្សេងទៀត។

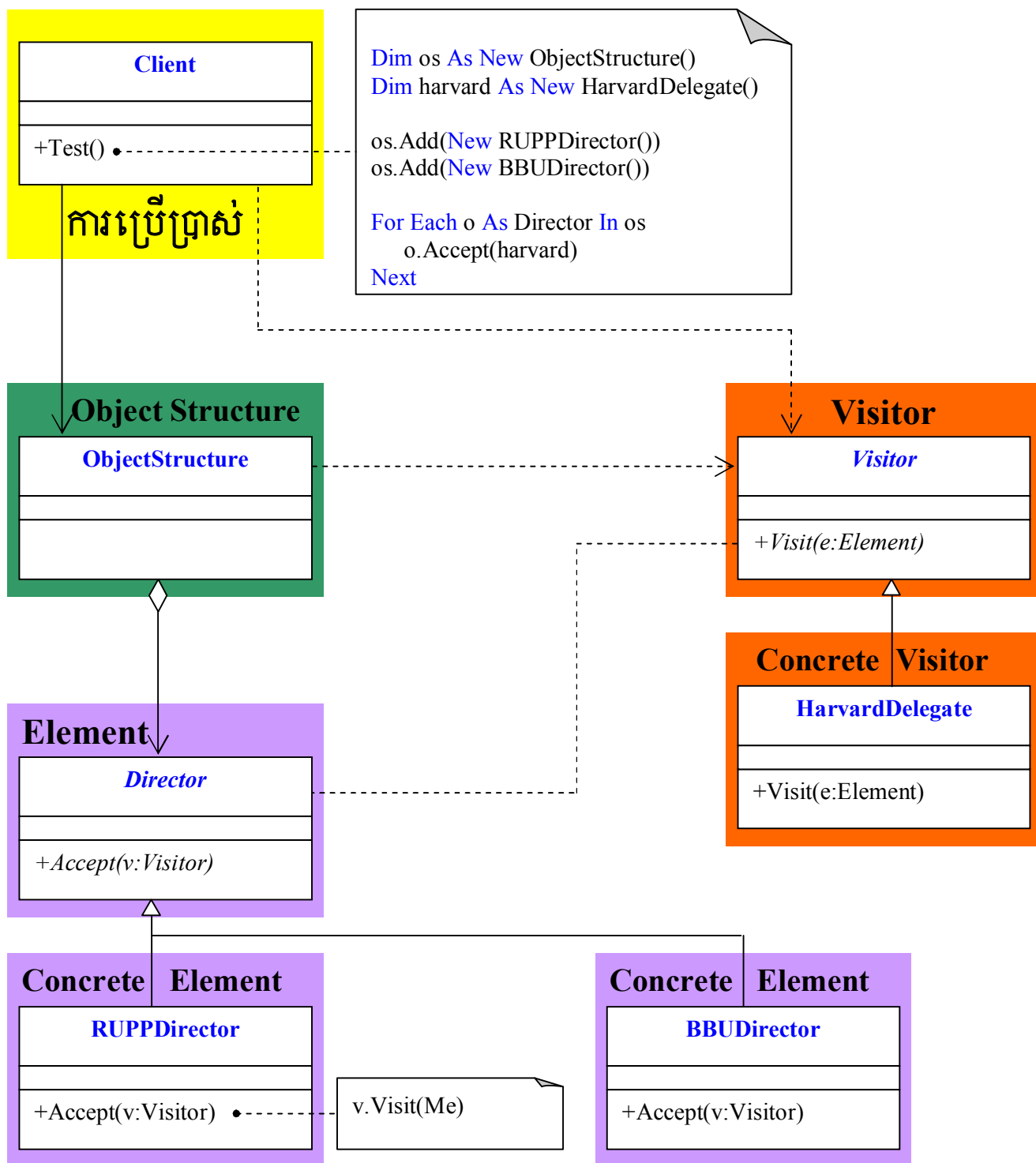


សាកលវិទ្យាល័យនានាអាចទទួលបានគណៈប្រតិភូផ្សេងៗទៀត។

ដូច្នេះ

គេអាចអនុវត្តនូវប្រតិបត្តិការថ្មីៗ ដោយពុំប៉ះពាល់ដល់ Class ជាតួអង្គផ្ទុកនូវលក្ខណៈទាំងឡាយ។

ដ្យាក្រាម UML នៃគំរូបែបទេសចរណ៍



Module Visitor

Visitor

```
Public Class Visitor
    Public Overridable Sub Visit(ByRef e As Director)
    End Sub
End Class
```

Concrete Visitor

```
Public Class HarvardDelegate
    Inherits Visitor
    Public Overrides Sub Visit(ByRef e As Director)
        Console.WriteLine("Harvard Delegate Thanks to " & e.ToString())
    End Sub
End Class
```

Element

```
Public Class Director
    Public Sub Accept(ByRef v As Visitor)
        v.Visit(Me)
    End Sub
    Public Overrides Function ToString() As String
        Return ""
    End Function
End Class
```

Concrete Element

```
Public Class RUPPDirector
    Inherits Director

    Public Overrides Function ToString() As String
        Return "Director of RUPP."
    End Function
End Class
```

Concrete Element

```
Public Class BBUDirector
    Inherits Director

    Public Overrides Function ToString() As String
        Return "Director of BBU."
    End Function
End Class
```

Object Structure

```
Public Class ObjectStructure
    Inherits List(Of Director)
End Class
```

Client

```
Public Class Client
    Public Shared Sub Test()
        Dim os As New ObjectStructure()
        Dim harvard As New HarvardDelegate()
```

```
os.Add(New RUPPDirector())  
os.Add(New BBUDirector())
```

```
For Each o As Director In os  
    o.Accept(harvard)  
Next
```

```
Console.Read()
```

```
End Sub
```

```
End Class
```

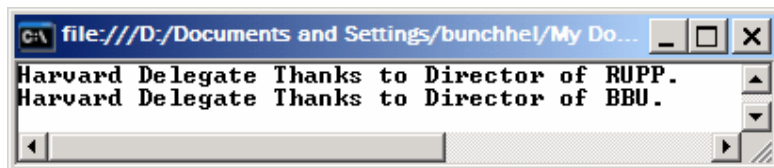
```
Sub Main()
```

```
    Client.Test()
```

```
End Sub
```

```
End Module
```

លទ្ធផល



គំរូបែបបំណាចក្រ

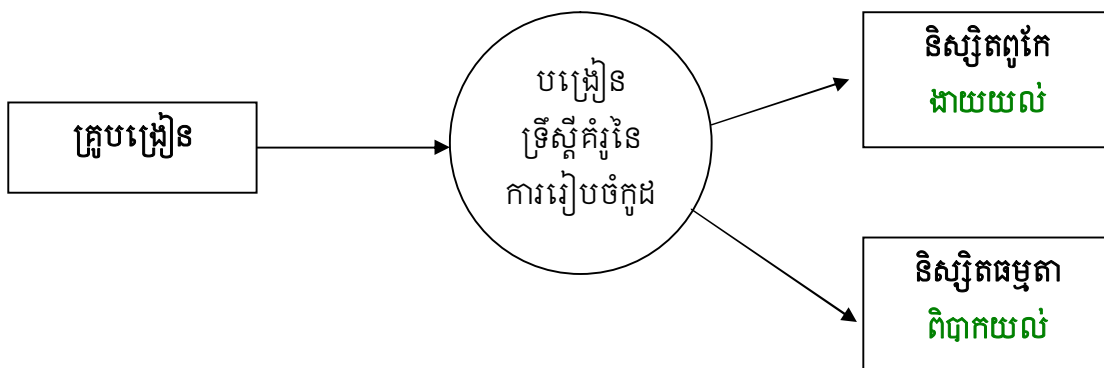
(Interpreter Pattern)

គោលដៅ

បំប្លែងទិន្នន័យពីទម្រង់មួយទៅទម្រង់ផ្សេងៗទៀត។

ជីវិតពិត

គ្រូបង្រៀនទ្រឹស្តីមួយដល់និស្សិតដែលមានកម្រិតយល់ដឹងខុសគ្នា។

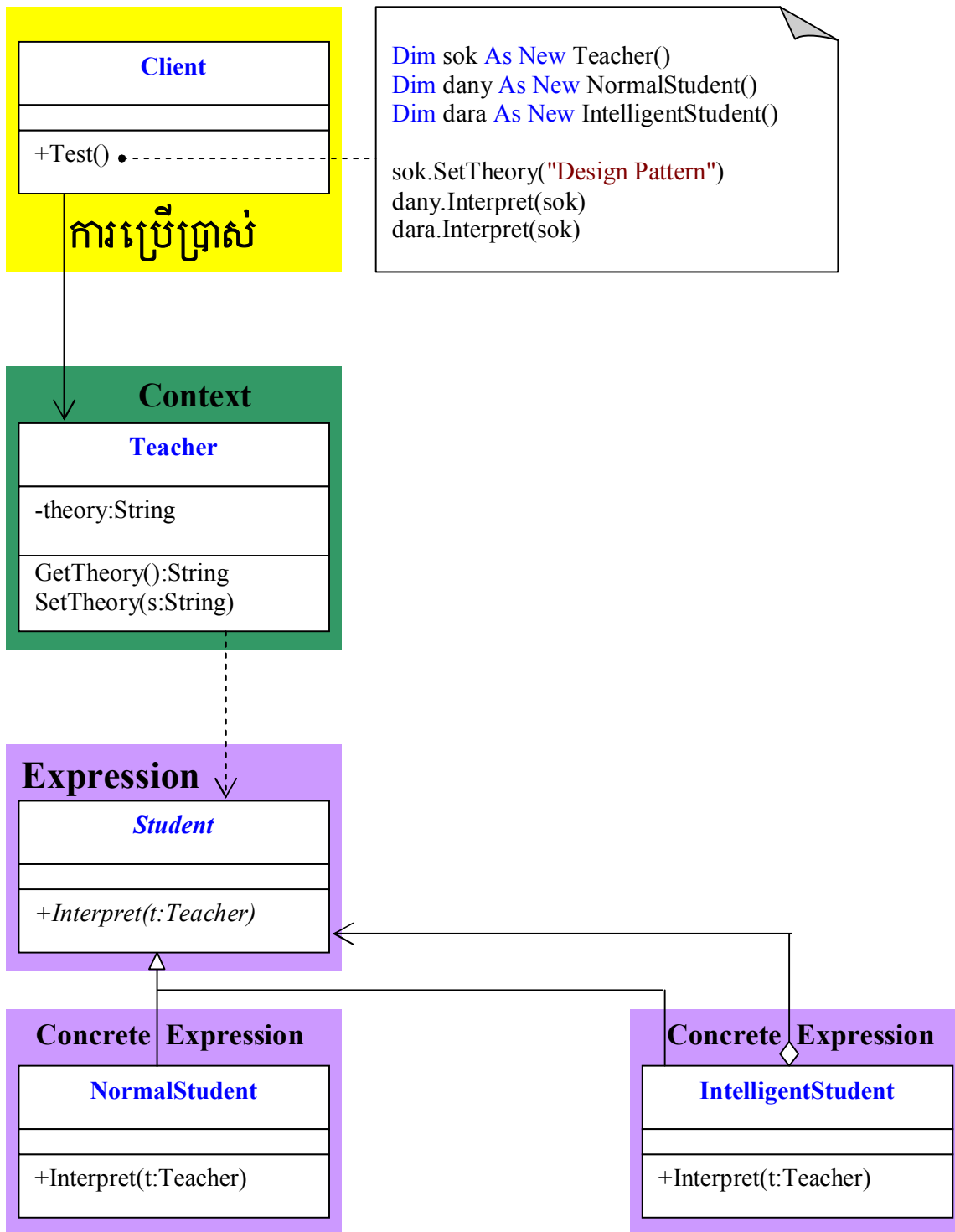


ទ្រឹស្តីមួយប៉ុន្តែការយល់និងបកស្រាយផ្សេងពីគ្នាអាស្រ័យកម្រិតយល់ដឹងរបស់និស្សិត។

ដូច្នេះ

ទិន្នន័យតែមួយប៉ុន្តែអាចបំប្លែងទៅជាទម្រង់ផ្សេងៗគ្នា។

ដ្យាក្រាម UML នៃគំរូបែបបំណាចក្រ



Module Interpreter

'Context

```
Public Class Teacher
    Private theory As String

    Public Function GetTheory() As String
        Return theory
    End Function

    Public Sub SetTheory(ByVal s As String)
        theory = s
    End Sub
End Class
```

'Expression

```
Public Class Student
    Public Overridable Sub Interpret(ByRef t As Teacher)
    End Sub
End Class
```

'Concrete Expression

```
Public Class NormalStudent
    Inherits Student
    Public Overrides Sub Interpret(ByRef t As Teacher)
        Console.WriteLine(t.GetTheory() & " is hard to understand.")
    End Sub
End Class
```

'Concrete Expression

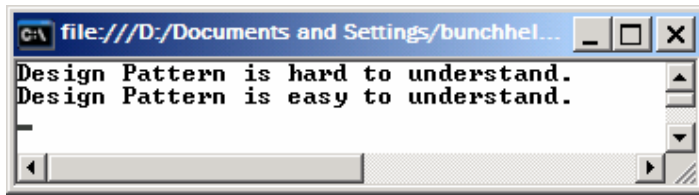
```
Public Class IntelligentStudent
    Inherits Student
    Public Overrides Sub Interpret(ByRef t As Teacher)
        Console.WriteLine(t.GetTheory() & " is easy to understand.")
    End Sub
End Class
```

```
Public Class Client
    Public Shared Sub Test()
        Dim sok As New Teacher()
        Dim dany As New NormalStudent()
        Dim dara As New IntelligentStudent()

        sok.SetTheory("Design Pattern")
        dany.Interpret(sok)
        dara.Interpret(sok)
        Console.Read()
    End Sub
End Class
```

```
Sub Main()
    Client.Test()
End Sub
End Module
```

លទ្ធផល



គំរូបែបសង្វាក់ទំនួលខុសត្រូវ

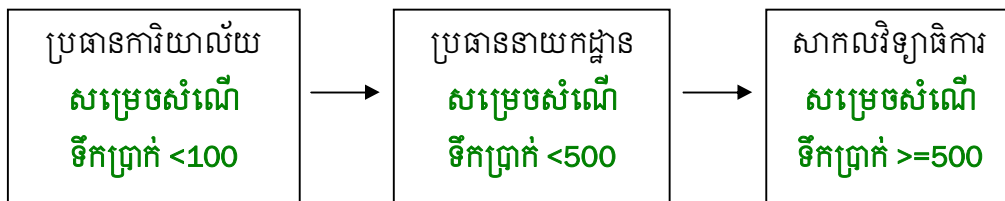
(Chain of Responsibility Pattern)

គោលដៅ

ប្រគល់ការទំនួលខុសត្រូវទៅ Class ម្តងមួយៗ នៅក្នុងខ្សែសង្វាក់នៃការងារ ដើម្បីប្រតិបត្តិ (Process)។

ជីវិតពិត

រៀបចំសំណើទៅថ្នាក់ដឹកនាំតាមឋានានុក្រមដើម្បីសុំការសម្រេច។

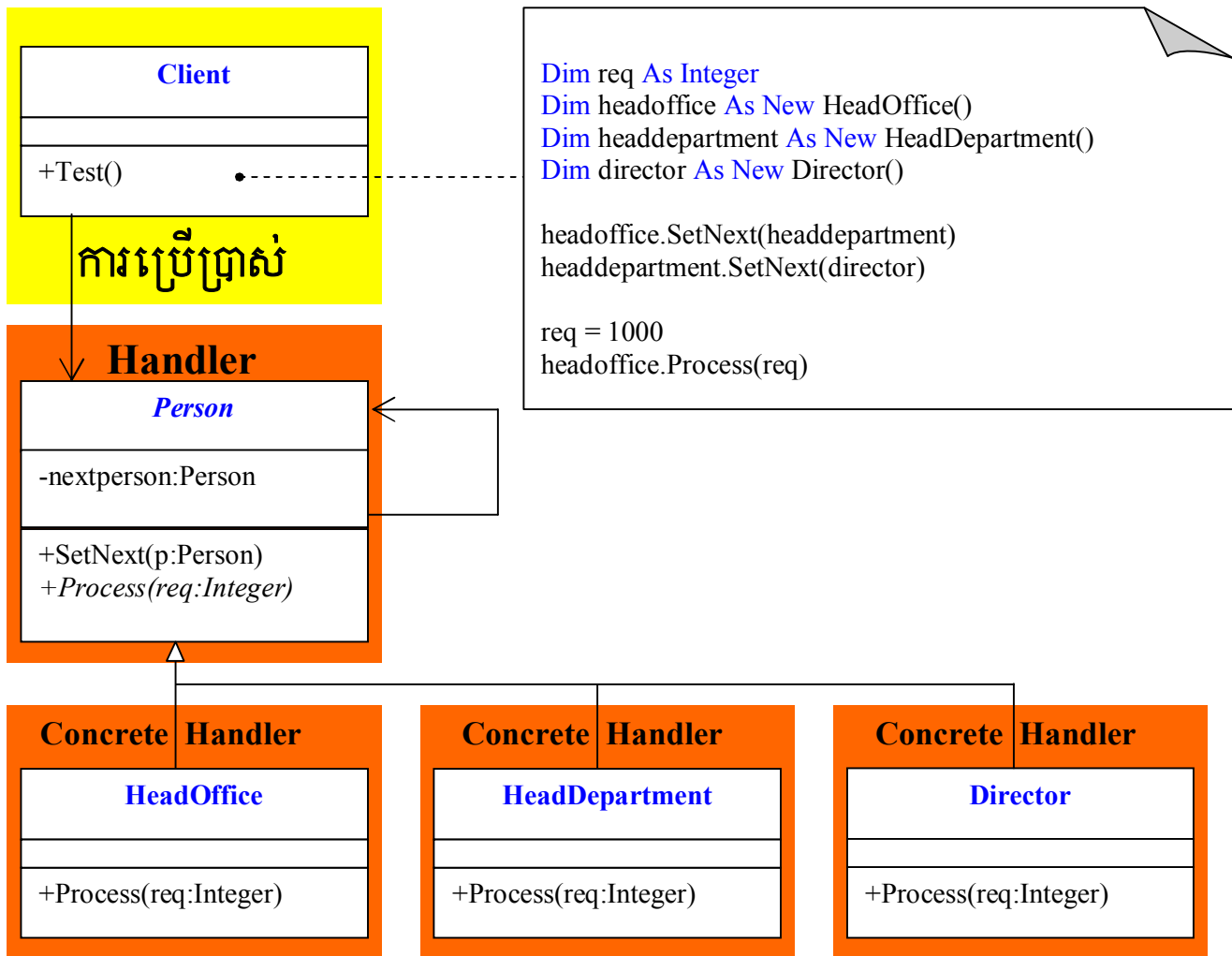


សំណើអាចត្រូវបានឆ្លងតាមឋានានុក្រមកំណត់តិច ឬ ច្រើនថ្នាក់ជាងនេះ។

ដូច្នេះ

ការងារអាចត្រូវបានដាក់ឱ្យឆ្លងកាត់ខ្សែសង្វាក់ទំនួលខុសត្រូវនៃ Class ឱ្យប្រតិបត្តិតាមការកំណត់។

ដ្យាក្រាម UML នៃគំរូបែបសង្វាក់ទំនួលខុសត្រូវ



Module ChainOfResponsibility

'Handler

```
Public Class Person
    Protected nextperson As Person = Nothing

    Public Sub SetNext(ByRef p As Person)
        nextperson = p
    End Sub

    Public Overridable Sub Process(ByVal req As Integer)
    End Sub
End Class
```

'Concrete Handler

```
Public Class HeadOffice
    Inherits Person

    Public Overrides Sub Process(ByVal req As Integer)
        If req < 100 Then
            Console.WriteLine("Head Office Approved " & req)
        Else
            Console.WriteLine("Head Office Approval Limit: <100")
            Console.WriteLine("Request will be sent to the Head Department for Approval")
            If Not nextperson Is Nothing Then
                nextperson.Process(req)
            End If
        End If
    End Sub
End Class
```

'Concrete Handler

```
Public Class HeadDepartment
    Inherits Person

    Public Overrides Sub Process(ByVal req As Integer)
        If req < 500 Then
            Console.WriteLine("Head Department Approved " & req)
        Else
            Console.WriteLine("Head Department Approval Limit: <500")
            Console.WriteLine("Request will be sent to the Director for Approval")
            If Not nextperson Is Nothing Then
                nextperson.Process(req)
            End If
        End If
    End Sub
End Class
```

'Concrete Handler

```
Public Class Director
    Inherits Person

    Public Overrides Sub Process(ByVal req As Integer)
```

```

        Console.WriteLine("Director Approved " & req)
    End Sub
End Class

Public Class Client
    Public Shared Sub Test()
        Dim req As Integer
        Dim headoffice As New HeadOffice()
        Dim headdepartment As New HeadDepartment()
        Dim director As New Director()

        'ទឹកប្រាក់តូចជាង ១០០ ប្រធានការិយាល័យសម្រេច មិនដូច្នោះត្រូវបញ្ជូន
        'សំណើបន្តទៅប្រធាននាយកដ្ឋាន
        headoffice.SetNext(headdepartment)

        'ទឹកប្រាក់តូចជាង ៥០០ ប្រធាននាយកដ្ឋានសម្រេច មិនដូច្នោះត្រូវបញ្ជូន
        'សំណើបន្តទៅសាកលវិទ្យាធិការ
        headdepartment.SetNext(director)

        req = 10
        Console.WriteLine("[Request of " & req & "]")
        headoffice.Process(req)

        req = 200
        Console.WriteLine()
        Console.WriteLine("[Request of " & req & "]")
        headoffice.Process(req)

        req = 1000
        Console.WriteLine()
        Console.WriteLine("[Request of " & req & "]")
        headoffice.Process(req)

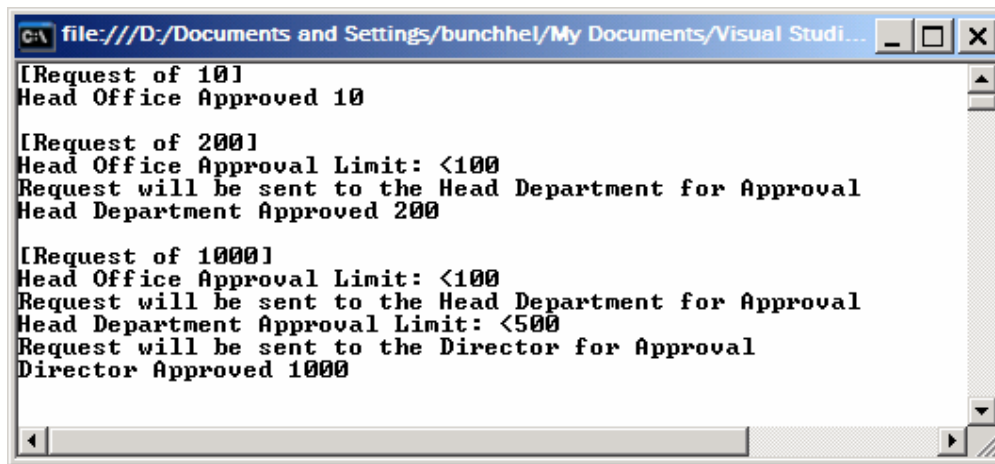
        Console.Read()
    End Sub
End Class

Sub Main()
    Client.Test()
End Sub

End Module

```

លទ្ធផល



```
file:///D:/Documents and Settings/bunchhel/My Documents/Visual Studi...
[Request of 10]
Head Office Approved 10

[Request of 200]
Head Office Approval Limit: <100
Request will be sent to the Head Department for Approval
Head Department Approved 200

[Request of 1000]
Head Office Approval Limit: <100
Request will be sent to the Head Department for Approval
Head Department Approval Limit: <500
Request will be sent to the Director for Approval
Director Approved 1000
```

គំរូបែបសន្លាកម្ម

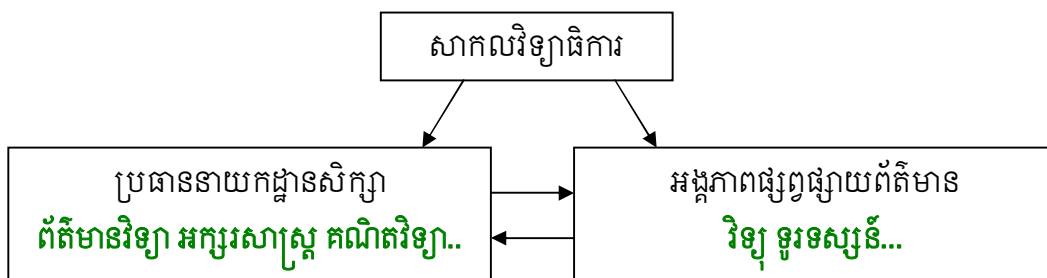
(Model View Controller Pattern)

គោលដៅ

ផ្តាច់ចេញពីគ្នារវាង Model (Business Logic) និង View (Interface) ដោយមាន Controller ជាអ្នកសម្របសម្រួល (សន្ធានាការី)។

ជីវិតពិត

សាកលវិទ្យាធិការបញ្ជាមកប្រធាននាយកដ្ឋានសិក្សានីមួយៗផ្សព្វផ្សាយពាណិជ្ជកម្មមហាវិទ្យាល័យដែលខ្លួនគ្រប់គ្រងរៀងៗខ្លួនតាមអង្គភាពផ្សព្វផ្សាយព័ត៌មាន។

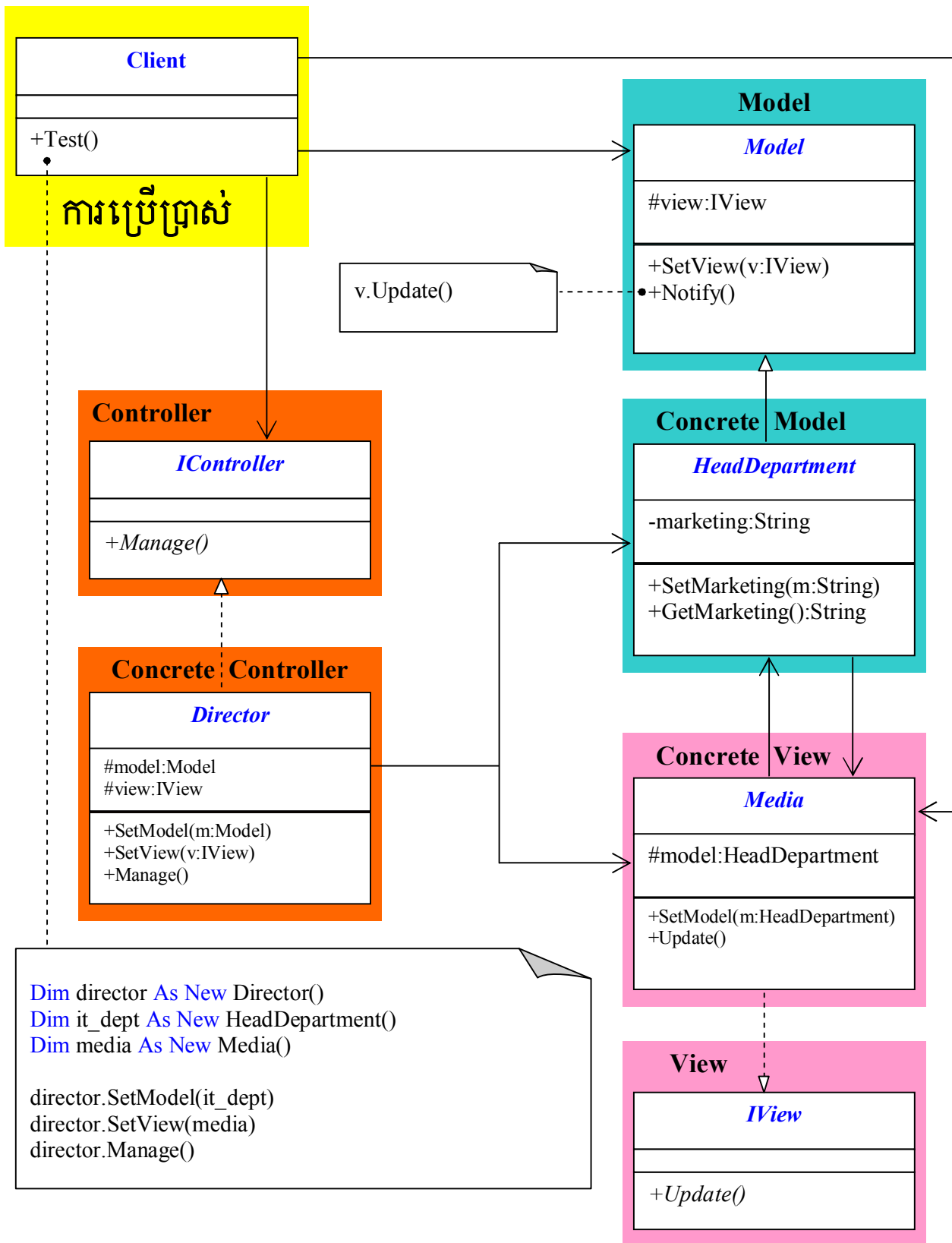


សាកលវិទ្យាធិការអាចបញ្ជាការងារផ្សេងៗទៀតស្របតាមផែនការខ្លួន ដោយប្រគល់ភារកិច្ចទៅប្រធាននាយកដ្ឋានសិក្សាជាអ្នករៀបចំបន្ត ដោយគ្រាន់តែចង្អុលទិសដៅសម្រាប់អនុវត្ត។

ដូច្នេះ

សំណូមពរពី Client ត្រូវបាន Controller សម្របសម្រួលដោះស្រាយដោយធ្វើការជាមួយ Model និង View សមស្រប។

ដ្យាក្រាម UML នៃគំរូបេបសន្លាតកម្ម



Module ModelViewController

```

'Controller
Public Interface IController
    Sub Manage()
End Interface

'Concrete Controller
Public Class Director
    Implements IController

    Protected model As HeadDepartment
    Protected view As Media

    Public Sub SetModel(ByRef m As Model)
        model = m
    End Sub

    Public Sub SetView(ByRef v As IView)
        view = v
    End Sub

    Public Sub Manage() Implements IController.Manage
        model.SetView(view)
        view.SetModel(model)

        Dim about As String

        about = "Marketing about Teachers"
        Console.WriteLine("Director: " & about)
        model.SetMarketing(about)

        Console.WriteLine()
        about = "Marketing about Teaching Materials"
        Console.WriteLine("Director: " & about)
        model.SetMarketing(about)
    End Sub
End Class

'Model
Public Class Model
    Protected view As IView

    Public Overridable Sub SetView(ByRef v As IView)
        view = v
    End Sub

    Public Sub Notify()
        view.Update()
    End Sub
End Class

'Concrete Model

```

```

Public Class HeadDepartment
    Inherits Model

    Private marketing As String

    Public Sub SetMarketing(ByVal m As String)
        marketing = m
        Console.WriteLine("Head Department: Prepared " & marketing)
        Notify()
    End Sub

    Public Function GetMarketing() As String
        Return marketing
    End Function
End Class

```

```

' View
Public Interface IView
    Sub Update()
End Interface

```

```

'Concrete View
Public Class Media
    Implements IView
    Protected model As HeadDepartment

    Public Sub SetModel(ByRef m As HeadDepartment)
        model = m
    End Sub

    Public Sub Update() Implements IView.Update
        Console.WriteLine("Media: Show " & model.GetMarketing())
    End Sub
End Class

```

```

Public Class Client
    Public Shared Sub Test()
        Dim director As New Director()
        Dim it_dept As New HeadDepartment()
        Dim media As New Media()

        director.SetModel(it_dept)
        director.SetView(media)
        director.Manage()

        Console.Read()
    End Sub
End Class

```

```

Sub Main()
    Client.Test()
End Sub

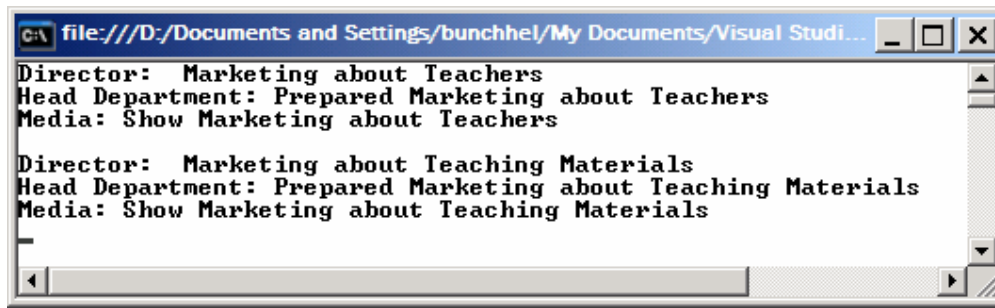
```

```

End Module

```

លទ្ធផល



វិទ្យាសាស្ត្រកម្មវិធី

ការស្វ័យសិក្សា

សុ ចិ បុ លី ប្រែថា ស្តាប់ គិត សួរ កត់។

ការតម្កល់ខ្លួនជាប្រយោជន៍ដល់ជនដទៃ

ធ្វើល្អបានល្អ

តម្កល់ចិត្តវិករាយចំពោះអ្វីដែលខ្លួនកំពុងមាន

នតិ សន្តិ បរិ សុខំ គ្មានសេចក្តីសុខអ្វីស្មើនឹងសេចក្តីស្ងប់។

ឯកសារពិគ្រោះ

1. *Design Patterns: Elements of Reusable Object-Oriented Software*, 1995 by Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides
2. http://en.wikipedia.org/wiki/Design_Patterns [ចូលថ្ងៃទី២៣ ខែតុលា ឆ្នាំ២០១៣]